



自然语言处理

7. 循环神经网络

虞剑飞

南京大学智能科学与技术学院

2025.4.2

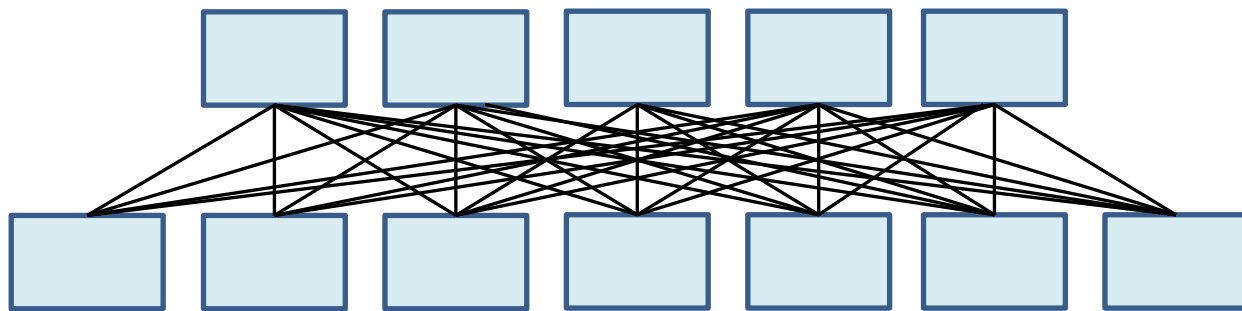
循环神经网络

- 引言
- 循环神经网络
- 参数学习
- 长程依赖问题
- 基于门控的循环神经网络
- 深度循环神经网络
- 递归神经网络
- 注意力机制
- 循环神经网络以及注意力机制的应用

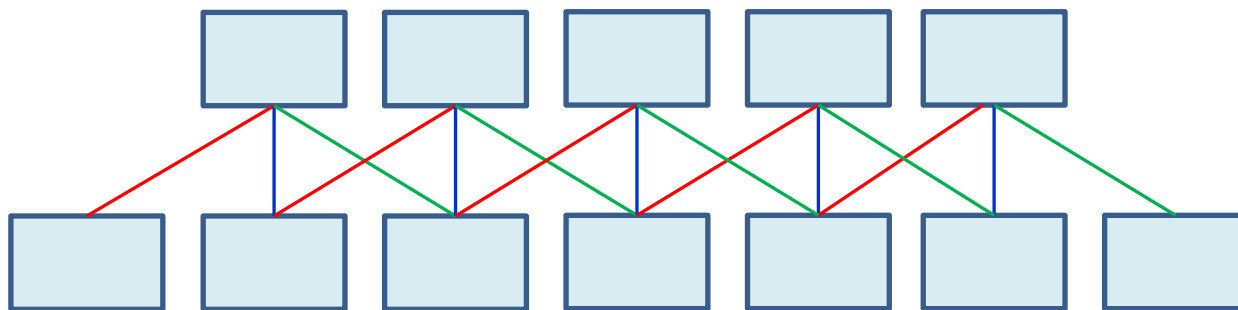
引言

- 前向神经网络

- 相邻两层之间存在单向连接，每层的节点之间是无连接的（无循环）
 - 输入和输出的维数都是固定的，不能任意改变
 - 无法处理变长序列数据



(a) 全连接层

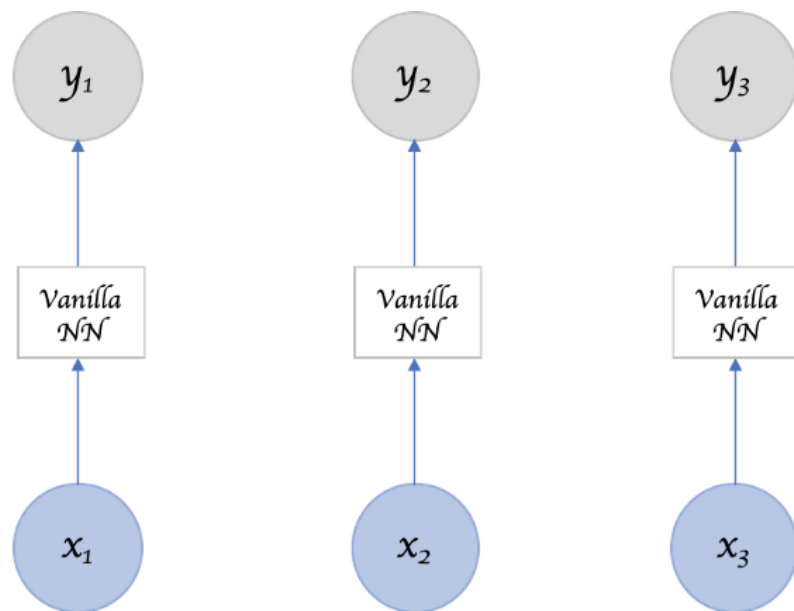


(b) 卷积层

引言

- 前向神经网络

- 假设每次输入都是独立的，也就是说每次网络的输出只依赖于当前的输入。

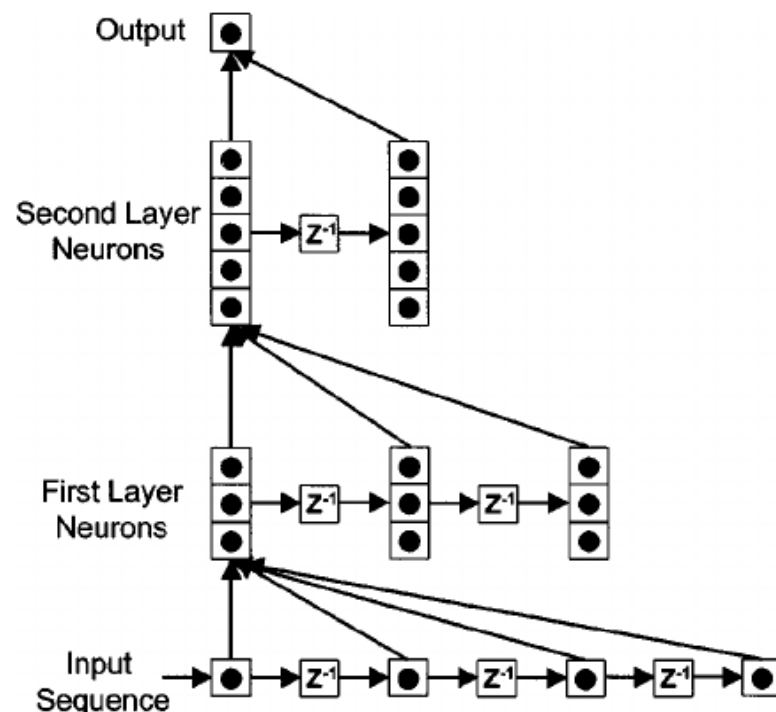


引言

- 如何给网络增加记忆能力？
 - 延时神经网络（Time Delay Neural Network, TDNN）
 - 建立一个额外的延时单元，用来存储网络的历史信息（可以包括输入、输出、隐状态等）

$$\mathbf{h}_t^{(l)} = f(\mathbf{h}_t^{(l-1)}, \mathbf{h}_{t-1}^{(l-1)}, \dots, \mathbf{h}_{t-K}^{(l-1)})$$

- 这样，前馈网络就具有了短期记忆的能力。

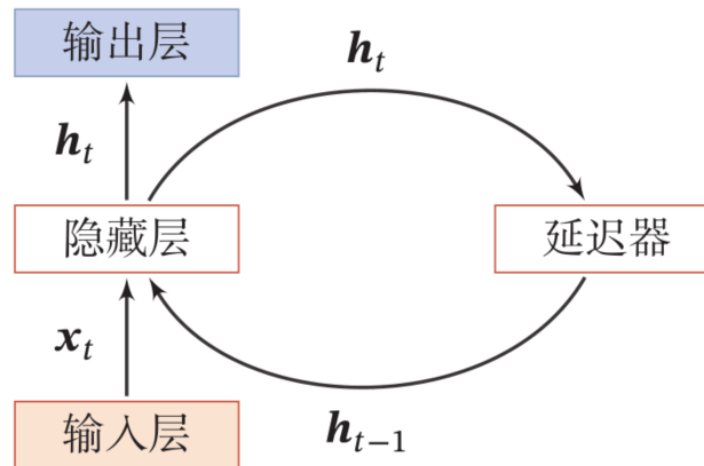


https://www.researchgate.net/publication/12314435_Neural_system_identification_model_of_human_sound_localization

循环神经网络

- 循环神经网络（Recurrent Neural Network, RNN）
 - 通过使用带自反馈的神经元，能够处理任意长度的时序数据。

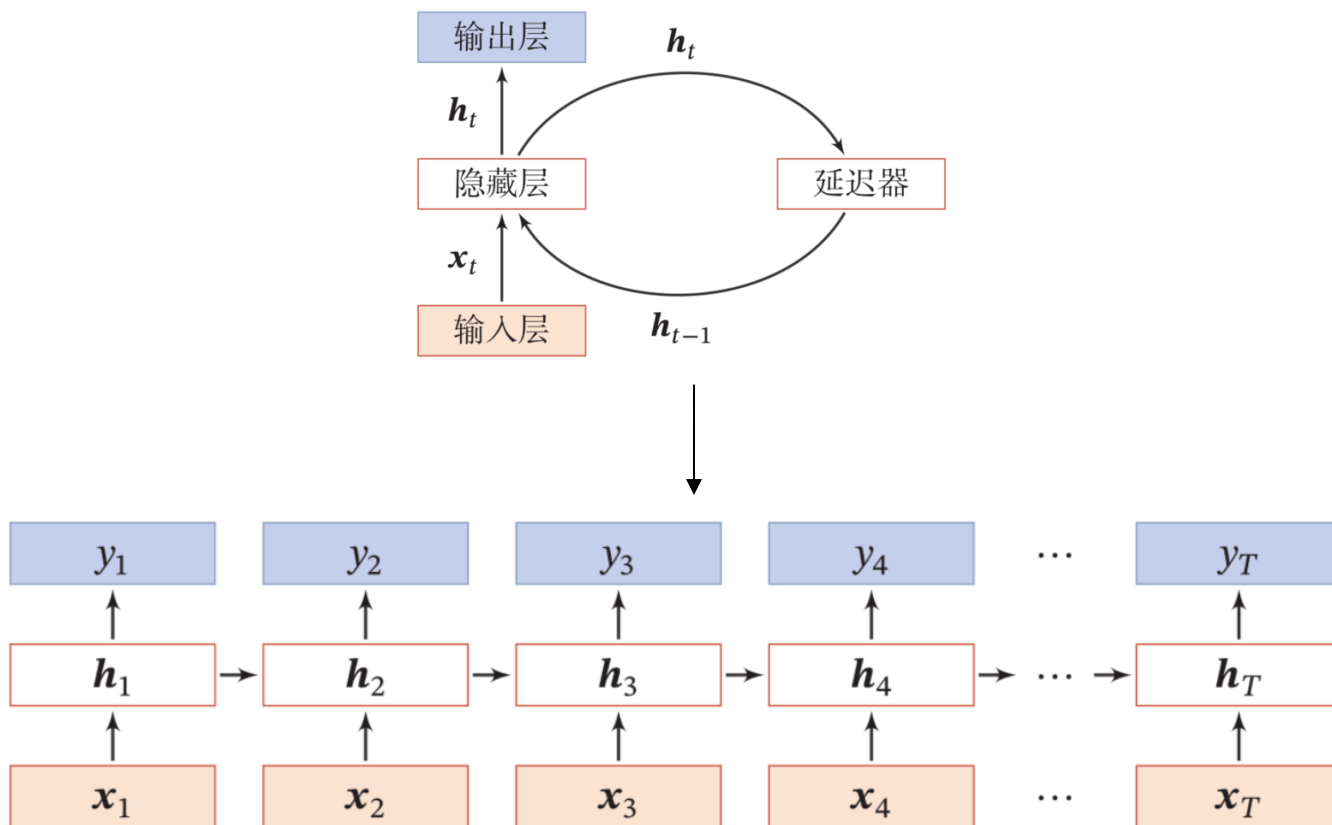
$$h_t = f(h_{t-1}, x_t)$$



- 循环神经网络比前馈神经网络更加符合生物神经网络的结构。
- 循环神经网络已经被广泛应用在语音识别、语言模型以及自然语言生成等任务上

循环神经网络

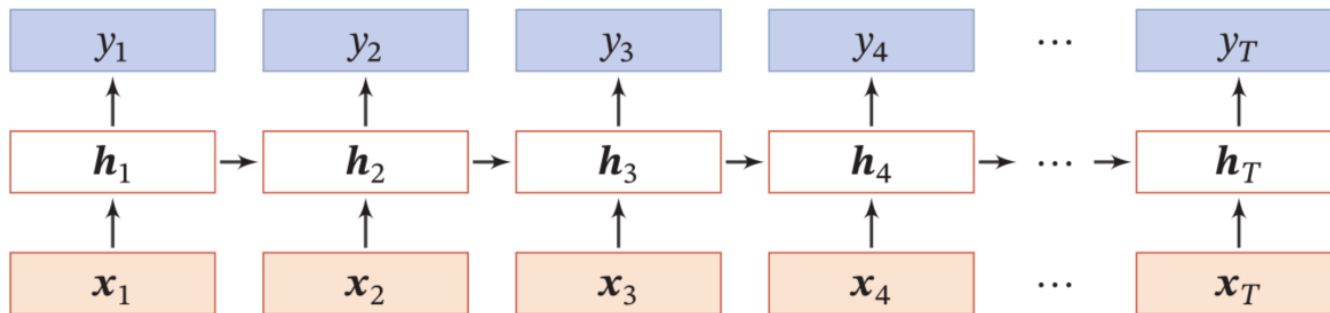
- 循环神经网络 (Recurrent Neural Network, RNN)
 - 按时间展开



循环神经网络

- 简单循环网络 (Simple Recurrent Neural Network, SRNN)
 - 状态更新:

$$\mathbf{h}_t = f(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{x}_t + \mathbf{b})$$

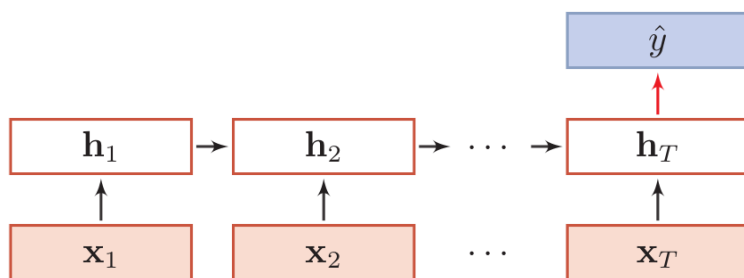


循环神经网络

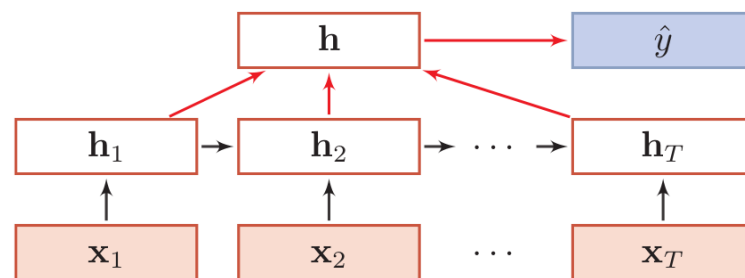
- 应用到基于机器学习的自然语言处理任务
 - 序列到类别
 - 同步的序列到序列模式
 - 异步的序列到序列模式

循环神经网络

- 应用到基于机器学习的自然语言处理任务
 - 序列到类别



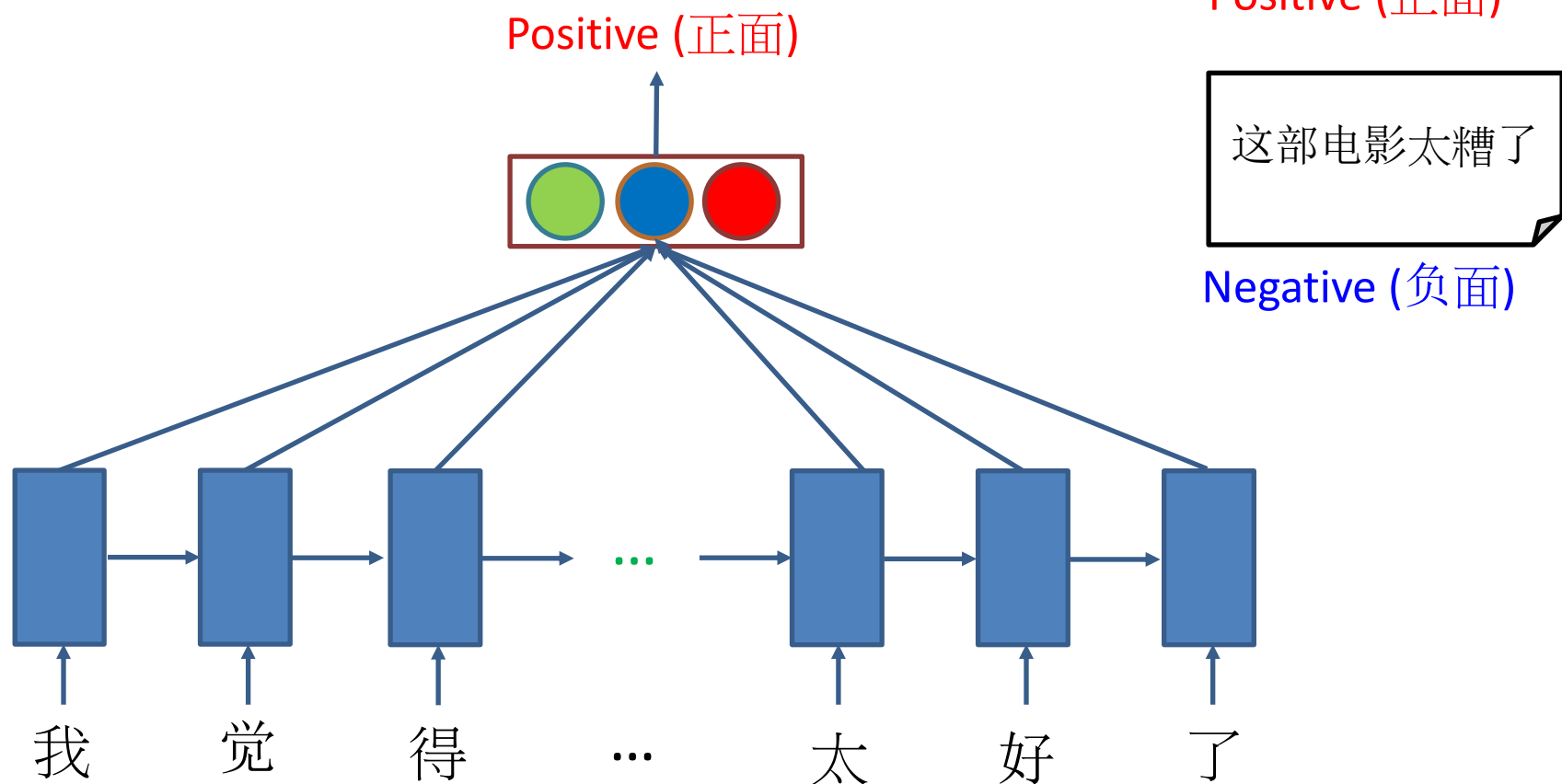
(a) 正常模式



(b) 按时间进行平均采样模式

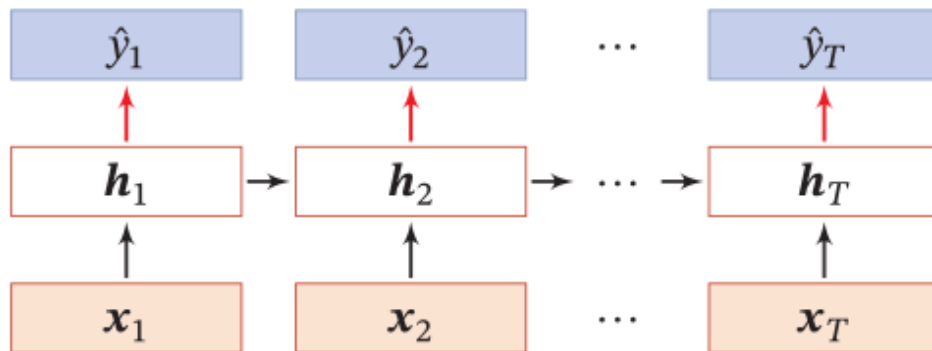
循环神经网络

- 应用到基于机器学习的自然语言处理任务
 - 例如情感分类



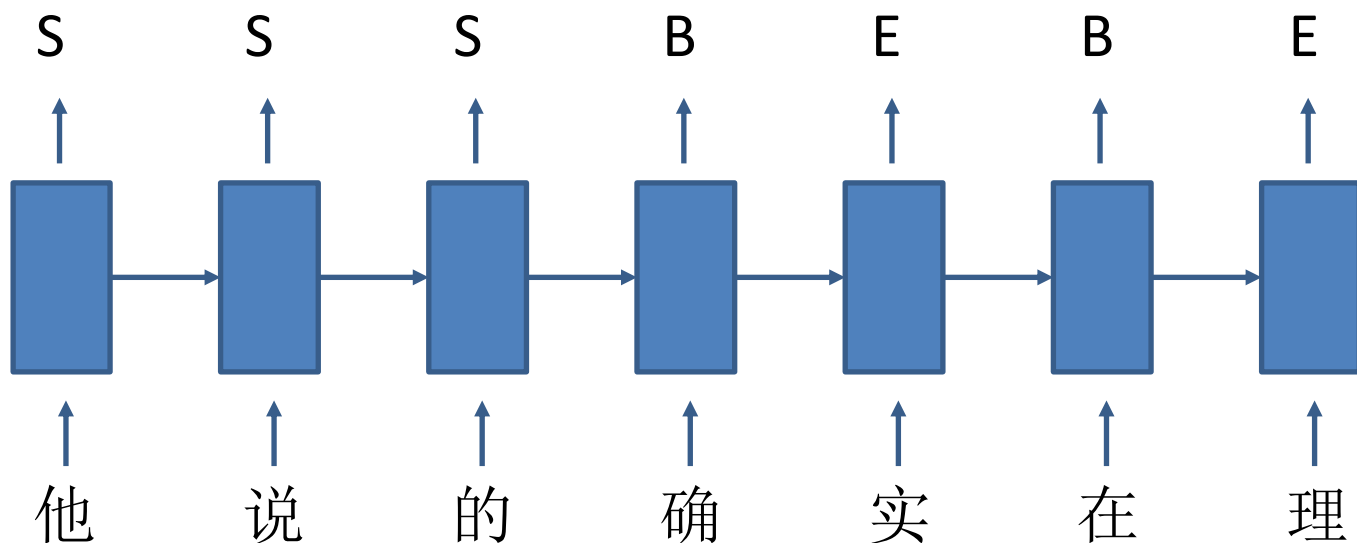
循环神经网络

- 应用到基于机器学习的自然语言处理任务
 - 同步的序列到序列模式



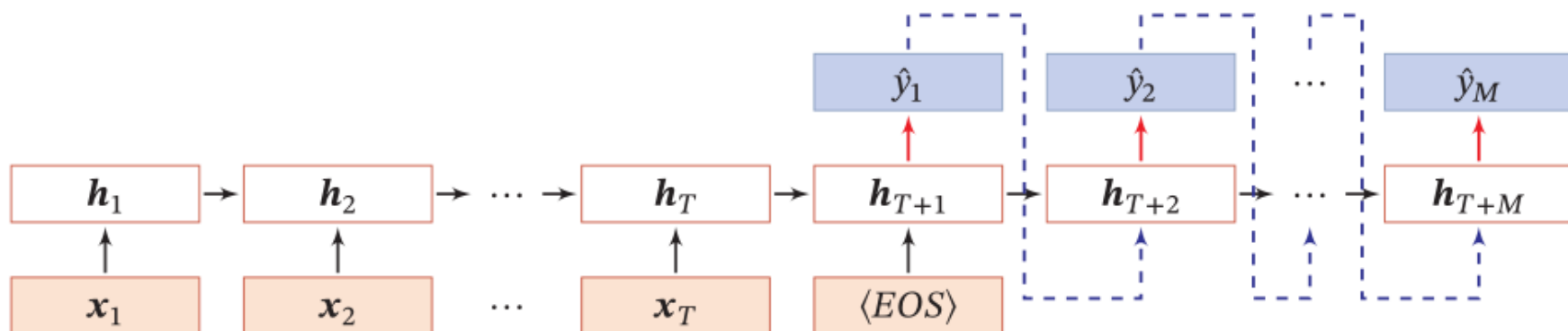
循环神经网络

- 应用到基于机器学习的自然语言处理任务
 - 例如中文分词



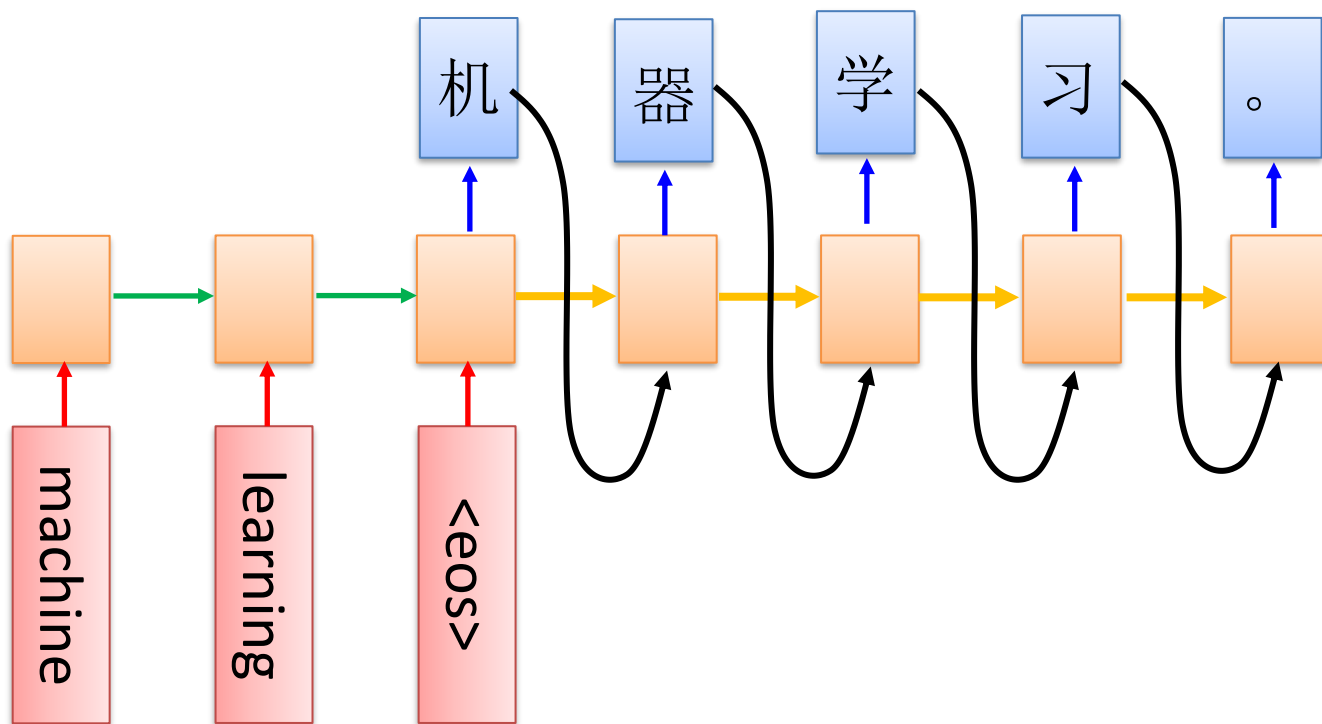
循环神经网络

- 应用到基于机器学习的自然语言处理任务
 - 异步的序列到序列模式



循环神经网络

- 应用到基于机器学习的自然语言处理任务
 - 例如机器翻译



循环神经网络

- 引言
- 循环神经网络
- **参数学习**
- **长程依赖问题**
- 基于门控的循环神经网络
- 深度循环神经网络
- 递归神经网络
- 注意力机制
- 循环神经网络的应用

循环神经网络

- 参数学习

- 给定一个训练样本 (x, y) ，其中

- $x = (x_1, \dots, x_T)$ 为长度是 T 的输入序列，
 - $y = (y_1, \dots, y_T)$ 是长度为 T 的标签序列。

- 时刻 t 的瞬时损失函数为 $\mathcal{L}_t = \mathcal{L}(y_t, g(h_t))$

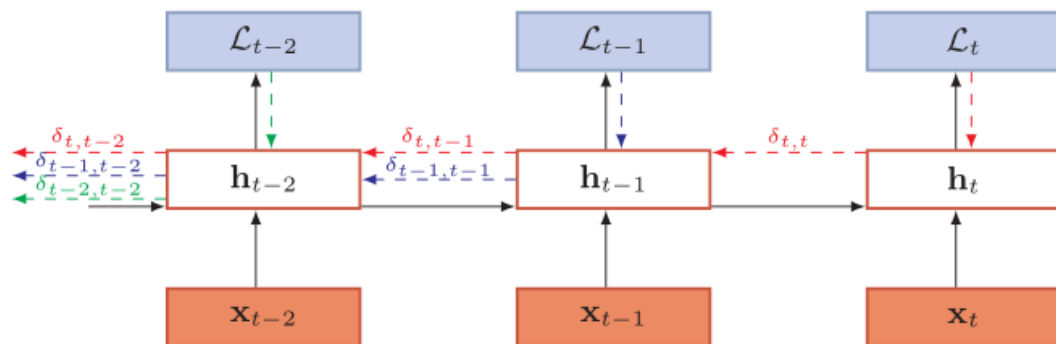
- 总损失函数为 $\mathcal{L} = \sum_{t=1}^T \mathcal{L}_t$

循环神经网络

- 参数学习

- 梯度：随时间反向传播算法 (Back Propagation Through Time, BPTT)

$$\mathbf{h}_t = f(\mathbf{z}_t); \mathbf{z}_t = \mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{x}_t + \mathbf{b}$$



$$\frac{\partial \mathcal{L}}{\partial \mathbf{U}} = \sum_{t=1}^T \sum_{k=1}^t \delta_{t,k} \mathbf{h}_{k-1}^T \quad \delta_{t,k} = \delta_{t,t} \prod_{\tau=k}^{t-1} (\mathbf{U}^T f'(\mathbf{z}_{\tau}))$$

$\delta_{t,k}$ 为第 t 时刻的损失对第 k 步隐藏神经元的净输入 $\mathbf{z}_k$ 的导数

循环神经网络

- 梯度消失/爆炸
 - 梯度

$$\frac{\partial \mathcal{L}}{\partial U} = \sum_{t=1}^T \sum_{k=1}^t \delta_{t,k} \mathbf{h}_{k-1}^T$$

- 其中

$$\begin{aligned} \delta_{t,k} &= \delta_{t,t} \prod_{\tau=k}^{t-1} \frac{(\mathbf{U}^T f'(\mathbf{z}_{\tau}))}{\gamma} \\ &\cong \delta_{t,t} \gamma^{t-k} \end{aligned}$$

由于梯度爆炸或消失问题，实际上只能学习到短周期的依赖关系。这就是所谓的长程依赖问题。

循环神经网络

- 长程依赖问题
 - 循环神经网络在时间维度上非常深
 - 梯度消失或梯度爆炸
- 如何改进？
 - 梯度爆炸问题
 - 权重衰减
 - 梯度截断
 - 梯度消失问题
 - 改进模型

循环神经网络

- 改进方法

- 循环边改为线性依赖关系

$$\mathbf{h}_t = f(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{x}_t + \mathbf{b})$$



$$\mathbf{h}_t = \mathbf{h}_{t-1} + \mathbf{W}\mathbf{x}_t + \mathbf{b}$$



$$\mathbf{h}_t = \mathbf{h}_{t-1} + \mathbf{g}(\mathbf{x}_t; \theta)$$

- 增加非线性

sum

$$\mathbf{h}_t = \mathbf{h}_{t-1} + \mathbf{g}(\mathbf{x}_t, \mathbf{h}_{t-1}; \theta)$$



为了h_t稳定

g 取 tanh

$$\begin{aligned}\delta_{t,k} &= \delta_{t,t} \prod_{\tau=k}^{t-1} \left(\frac{\mathbf{U}^T f'(\mathbf{z}_\tau)}{\gamma} \right) \\ &\cong \delta_{t,t} \gamma^{t-k}\end{aligned}$$

循环神经网络

- 引言
- 循环神经网络
- 参数学习
- 长程依赖问题
- 基于门控的循环神经网络
- 深度循环神经网络
- 递归神经网络
- 注意力机制
- 循环神经网络以及注意力机制的应用

基于门控的循环神经网络

- 门控机制
 - 长程依赖问题
 - 门控机制
 - 控制信息的累积速度，包括有选择地加入新的信息，并有选择地遗忘之前历史累计的信息
 - 基于门控机制的循环神经网络（Gated RNN）
 - 门控循环单元（Gated Recurrent Unit, GRU）
 - 长短期记忆神经网络（Long Short-Term Memory, LSTM）

基于门控的循环神经网络

- 门控循环单元 (Gated Recurrent Unit, GRU)

$$h_t = h_{t-1} + g(x_t, h_{t-1}; \theta)$$

$$h_t = \mathbf{z}_t \odot h_{t-1} + (1 - \mathbf{z}_t) \odot g(x_t, h_{t-1}; \theta)$$

更新门

$$\mathbf{z}_t = \sigma(W_z x_t + U_z h_{t-1} + b_z)$$

$$\tilde{h}_t = \tanh(W_h x_t + U_h h_{t-1} + b_h)$$

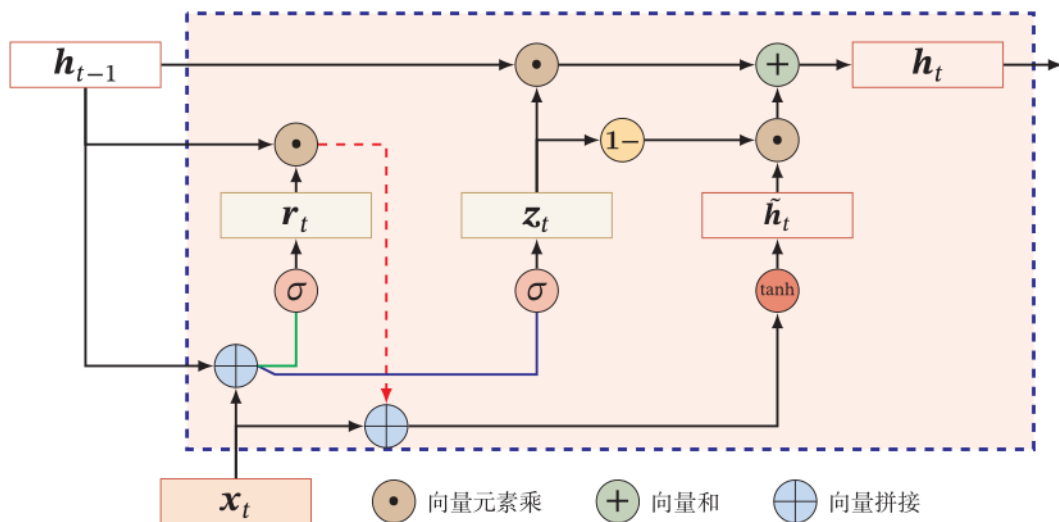
重置门

$$\mathbf{r}_t = \sigma(W_r x_t + U_r h_{t-1} + b_r)$$

$$\tilde{h}_t = \tanh(W_h x_t + U_h (\mathbf{r}_t \odot h_{t-1}) + b_h)$$

基于门控的循环神经网络

- 门控循环单元 (Gated Recurrent Unit, GRU)



重置门

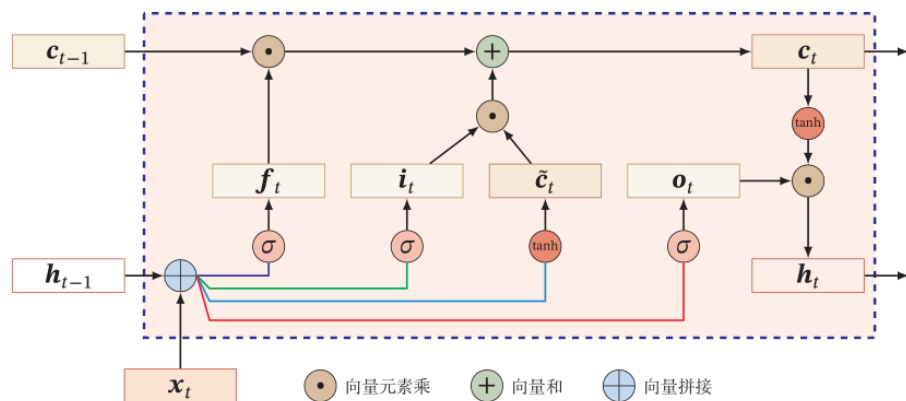
$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r) \quad \tilde{h}_t = \tanh(W_h x_t + U_h (r_t \odot h_{t-1}) + b_h)$$

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z) \quad h_t = z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t$$

更新门

基于门控的循环神经网络

- 长短期记忆神经网络（Long Short-Term Memory, LSTM）



$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$



$$h_t = o_t \odot \tanh(c_t)$$

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i)$$

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f)$$

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o)$$

基于门控的循环神经网络

- LSTM的各种变体
 - 没有遗忘门

$$\mathbf{c}_t = \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$$

- 耦合输入门和遗忘门

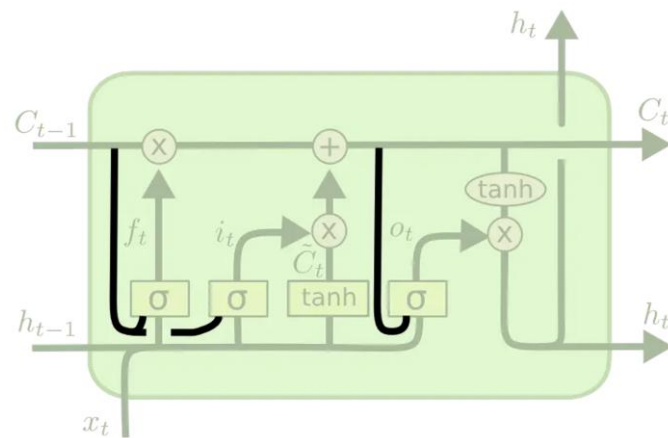
$$\mathbf{f}_t + \mathbf{i}_t = 1$$

- peephole连接

$$\mathbf{i}_t = \sigma(\mathbf{W}_i \mathbf{x}_t + \mathbf{U}_i \mathbf{h}_{t-1} + \mathbf{V}_i \mathbf{c}_{t-1} + \mathbf{b}_i)$$

$$\mathbf{f}_t = \sigma(\mathbf{W}_f \mathbf{x}_t + \mathbf{U}_f \mathbf{h}_{t-1} + \mathbf{V}_f \mathbf{c}_{t-1} + \mathbf{b}_f)$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_o \mathbf{x}_t + \mathbf{U}_o \mathbf{h}_{t-1} + \mathbf{V}_o \mathbf{c}_t + \mathbf{b}_o)$$

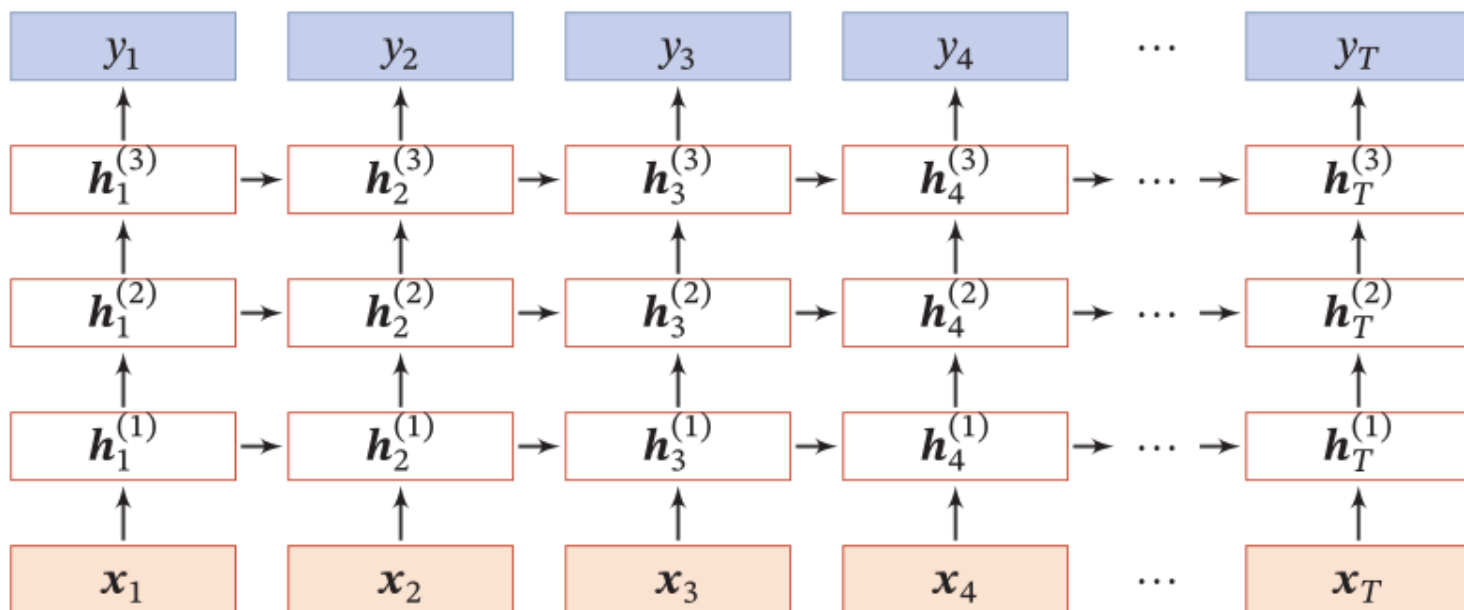


深度循环神经网络

- 引言
- 循环神经网络
- 参数学习
- 长程依赖问题
- 基于门控的循环神经网络
- 深度循环神经网络
- 递归神经网络
- 注意力机制
- 循环神经网络以及注意力机制的应用

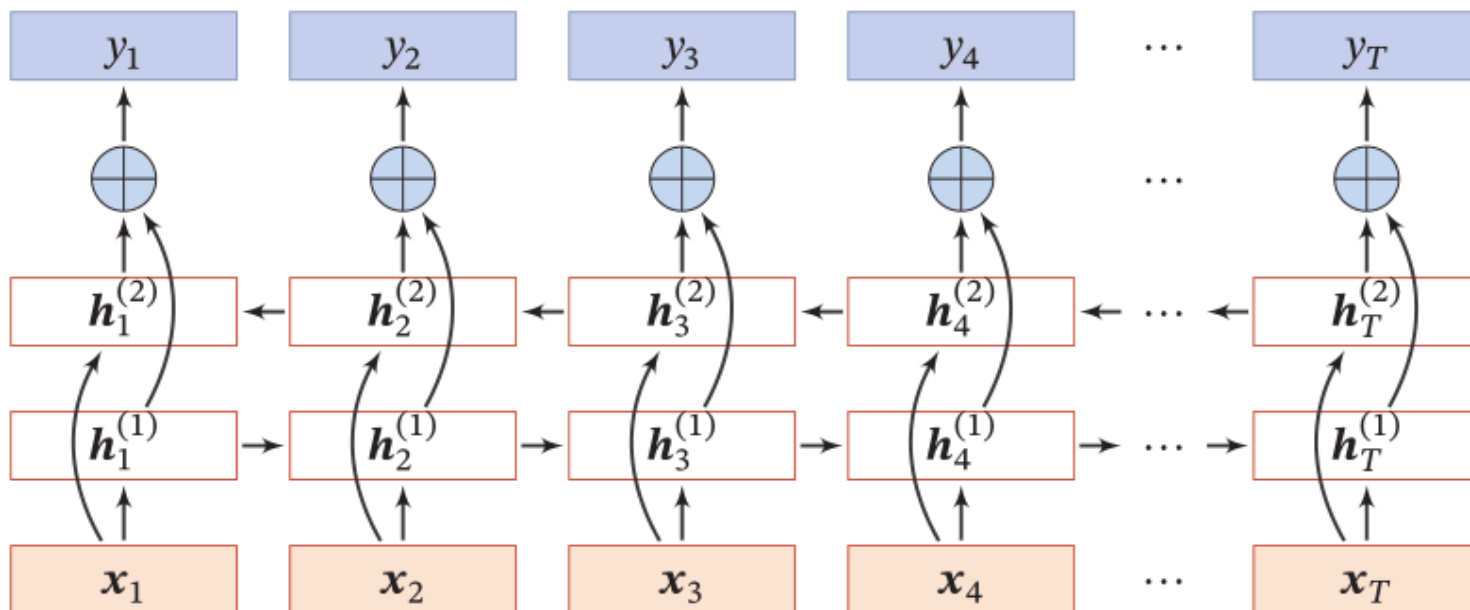
深度循环神经网络

- 堆叠循环神经网络



深度循环神经网络

- 双向循环神经网络



深度循环神经网络

- 代码
 - 情感分类 (RNN/GRU/LSTM)

```
class Classifier(nn.Module):
    """ A Classifier with an RNN to extract features and an MLP to classify """
    def __init__(self, embedding_size, num_embeddings, num_classes,
                  rnn_hidden_size, bidirectional=False, batch_first=True, padding_idx=0):
        super(Classifier, self).__init__()

        if bidirectional == False:
            self.num_directions = 1
        else:
            self.num_directions = 2

        self.emb = nn.Embedding(num_embeddings=num_embeddings,
                                embedding_dim=embedding_size,
                                padding_idx=padding_idx)

        #self.rnn = nn.RNN(input_size=embedding_size,
        self.rnn = nn.GRU(input_size=embedding_size,
        self.rnn = nn.LSTM(input_size=embedding_size,
                            hidden_size=rnn_hidden_size,
                            batch_first=batch_first,
                            num_layers = 1,
                            dropout = 0.0,
                            bidirectional=bidirectional)

        self.fc1 = nn.Linear(in_features=rnn_hidden_size*self.num_directions,
                              out_features=rnn_hidden_size*self.num_directions)
        self.fc2 = nn.Linear(in_features=rnn_hidden_size*self.num_directions,
                              out_features=num_classes)

        # for batch norm
        self.bn1 = nn.BatchNorm1d(rnn_hidden_size*self.num_directions)
```

```
def forward(self, x_in, x_lengths=None, apply_softmax=False):
    x_embedded = self.emb(x_in)

    y_out, _ = self.rnn(x_embedded)

    #y_out = y_out_last.squeeze(dim=0)

    if x_lengths is not None:
        #y_out = column_gather(y_out, x_lengths)
        # but, the last hidden vector of last
        # e.g., get the last hidden vector from
        # uncomment code below for task 2, and comment out the line above.
        # y_out gets the max or mean hidden vector of each input
        y_out = column_summation(y_out, x_lengths, mode="max")
    else:
        y_out = y_out[:, -1, :]

    # with batch norm and dropout
    y_out = F.relu(self.bn1(self.fc1(F.dropout(y_out, 0.2, training=self.training))))

    # with dropout
    y_out = self.fc2(F.dropout(y_out, 0.2, training=self.training))

    if apply_softmax:
        y_out = F.softmax(y_out, dim=1)

    return y_out
```

深度循环神经网络

- 代码
 - 情感分类

```
def forward(self, x_in, x_lengths=None, apply_softmax=False):
    x_embedded = self.emb(x_in) # (batch, seq_size) -> (batch, seq_size, feat_size)

    y_out, _ = self.rnn(x_embedded) # (batch, seq_size, feat_size) -> (batch, seq_size, hidden_size)
    # (64,19,100) -> (64,19,64*num_directions) # hidden_size = 64 * num_directions

    # y_out = y_out_last.squeeze(dim=0) # convert (1, batch, hidden_size*num_directions) to (batch, hidden_size*num_directions)

    if x_lengths is not None:
        # (batch, ) ; e.g., (64,) ; e.g., [9, 6, 11, 9, 7]
        # y_out = column_gather(y_out, x_lengths) # y_out gets the last hidden vector of each input
        # (64, 64*num_directions)
        # but, the last hidden vector of last input is not the last hidden vector of the batch
        # e.g., get the last hidden vector from each input
        # uncomment code below for task 2, and comment out the line above.
        # y_out gets the max or mean hidden vector of each input
        y_out = column_summation(y_out, x_lengths, mode="max") # used in assignment ???
    else:
        y_out = y_out[:, -1, :] # y_out gets the last hidden vector of each input
        # (64, 64*num_directions)

    # with batch norm and dropout
    y_out = F.relu(self.bn1(self.fc1(F.dropout(y_out, 0.2, training=self.training))))

    # with dropout
    y_out = self.fc2(F.dropout(y_out, 0.2, training=self.training)) #0.2 # y_out: (batch, hidden_size)

    if apply_softmax:
        y_out = F.softmax(y_out, dim=1)

    return y_out
```

```
def column_gather(y_out, x_lengths):
    x_lengths = x_lengths.long().detach().cpu().numpy() - 1 # deduct 1 to get the last hidden vector of each input
    out = []
    for batch_index, column_index in enumerate(x_lengths): # out gets the last hidden vector of each input
        out.append(y_out[batch_index, column_index]) # e.g., y_out[0, 9],
    return torch.stack(out) # (batch, hidden_size*num_directions); E.g.,
```

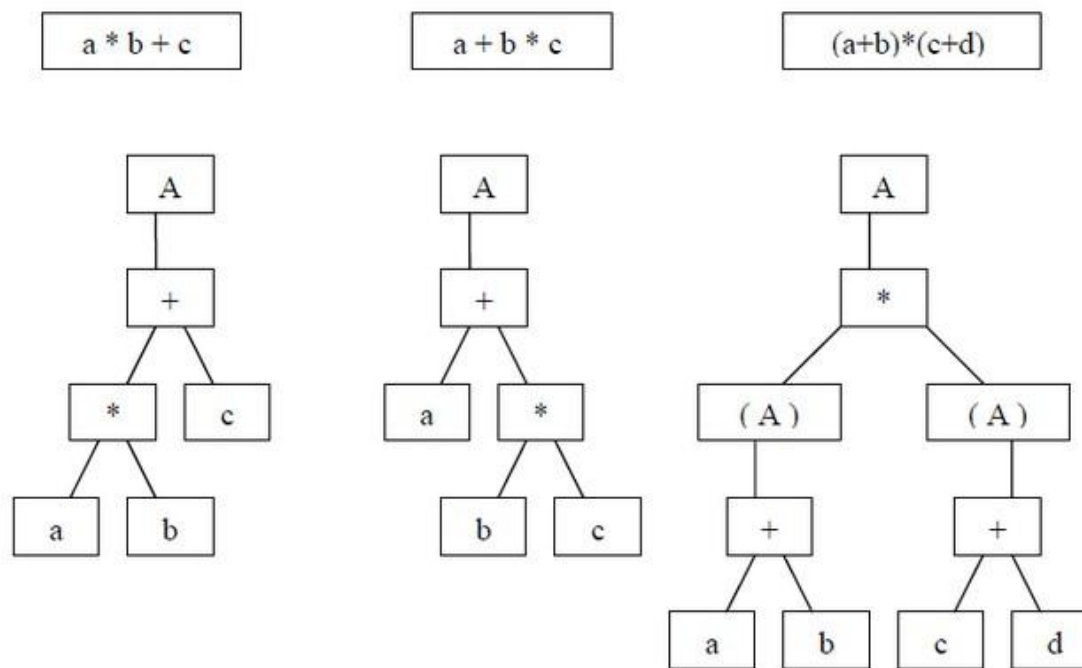
```
def column_summation(y_out, x_lengths, mode="mean"):
    x_lengths = x_lengths.long().detach().cpu().numpy() - 1
    out = []
    for batch_index, column_index in enumerate(x_lengths):
        if mode == "mean":
            avg = torch.mean(y_out[batch_index, :column_index], 0)
            out.append(avg)
        elif mode == "max":
            maximum, maximum_index = torch.max(y_out[batch_index, :column_index], 0)
            out.append(maximum)
        else:
            out.append(y_out[batch_index, column_index])
    return torch.stack(out)
```

递归神经网络

- 引言
- 循环神经网络
- 参数学习
- 长程依赖问题
- 基于门控的循环神经网络
- 深度循环神经网络
- **递归神经网络**
- 注意力机制
- 循环神经网络以及注意力机制的应用

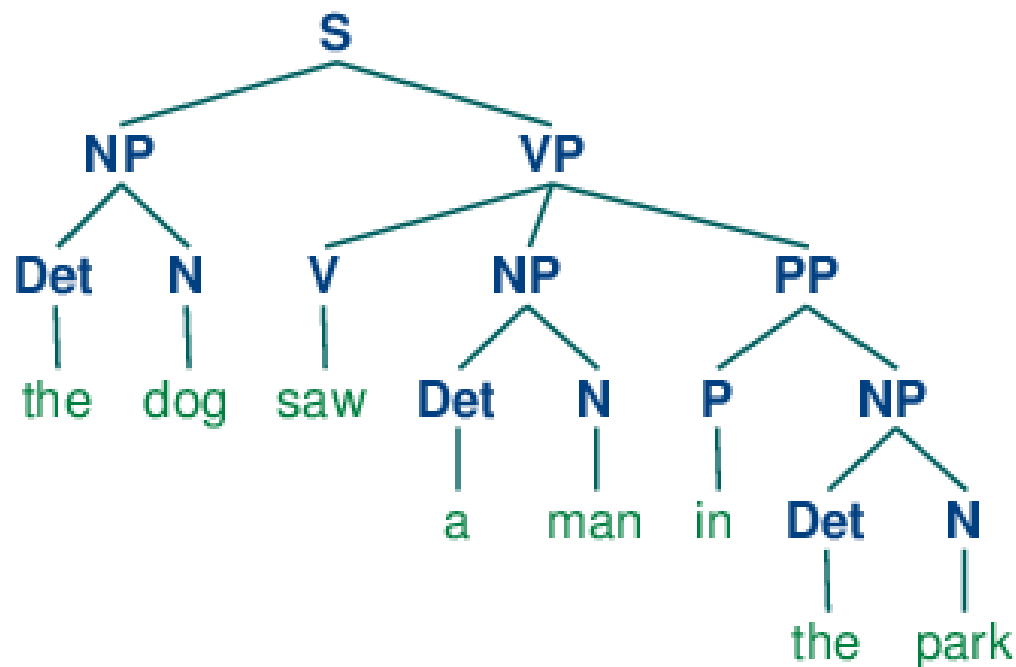
递归神经网络

- 循环神经网络
 - 用于处理任意长度的时序数据
 - 如何处理具有预定义结构的数据？
 - 程序语言的句法结构



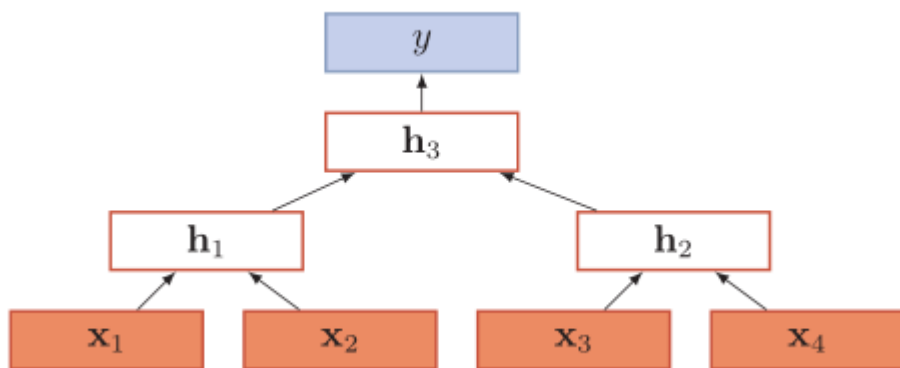
递归神经网络

- 循环神经网络
 - 用于处理任意长度的时序数据
 - 如何处理具有预定义结构的数据？
 - 自然语言的句法结构



递归神经网络

- 递归神经网络（Recursive Neural Network, RvNN）
 - Richard Socher, et al. 2011.



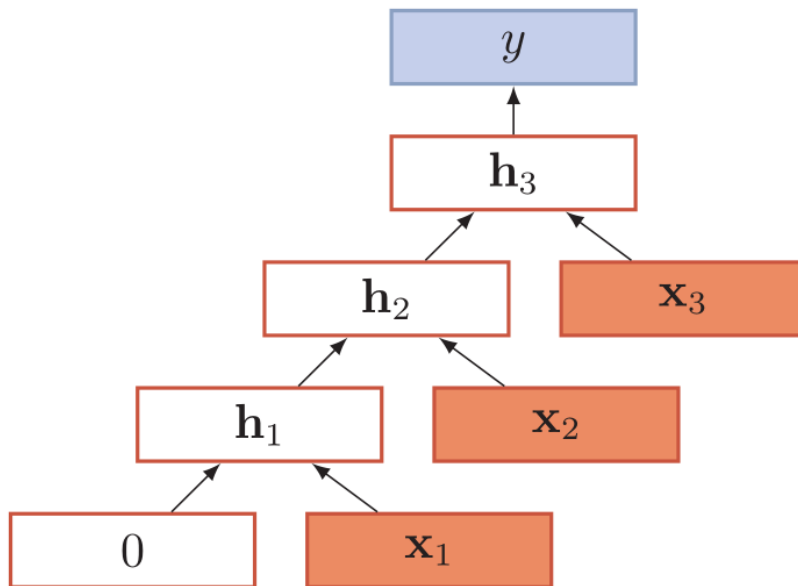
$$h_1 = f \left(W \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + b \right),$$

$$h_2 = f \left(W \begin{bmatrix} x_3 \\ x_4 \end{bmatrix} + b \right),$$

$$h_3 = f \left(W \begin{bmatrix} h_1 \\ h_2 \end{bmatrix} + b \right)$$

递归神经网络

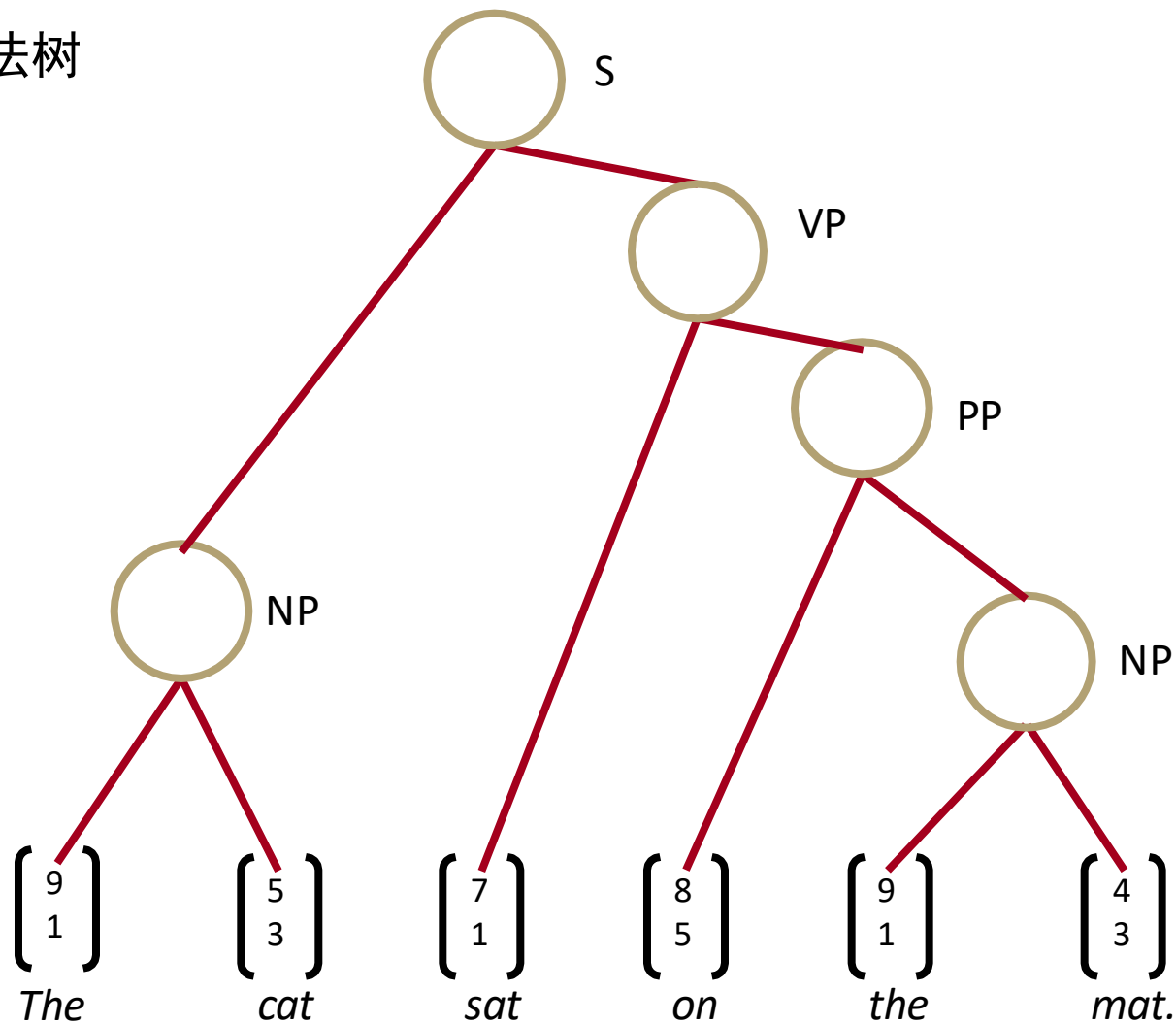
- 退化为循环神经网络
 - 循环神经网络是递归神经网络的特例



$$h_t = f(h_{t-1}, x_t)$$

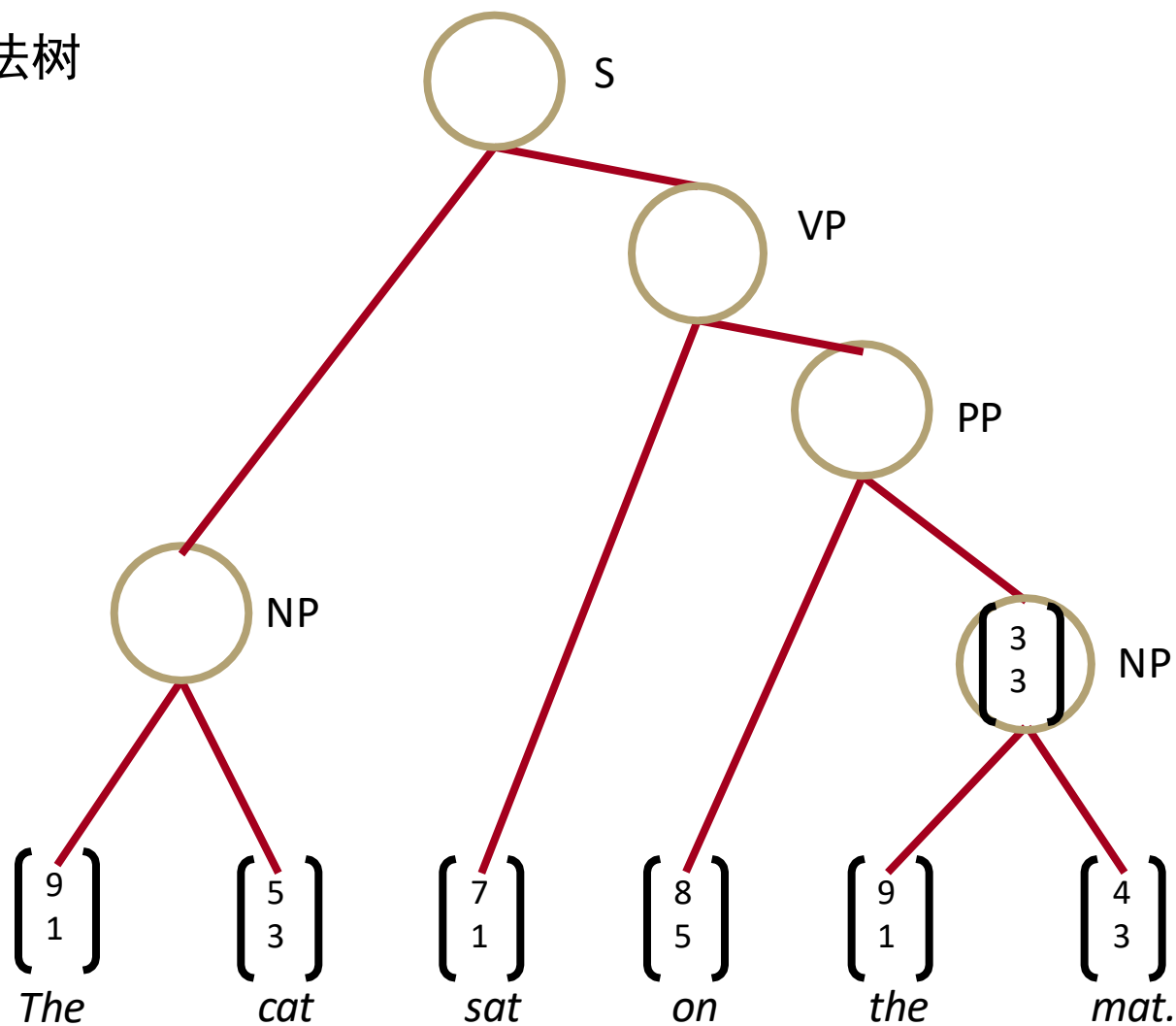
递归神经网络

- 示例
 - 给定一个语法树



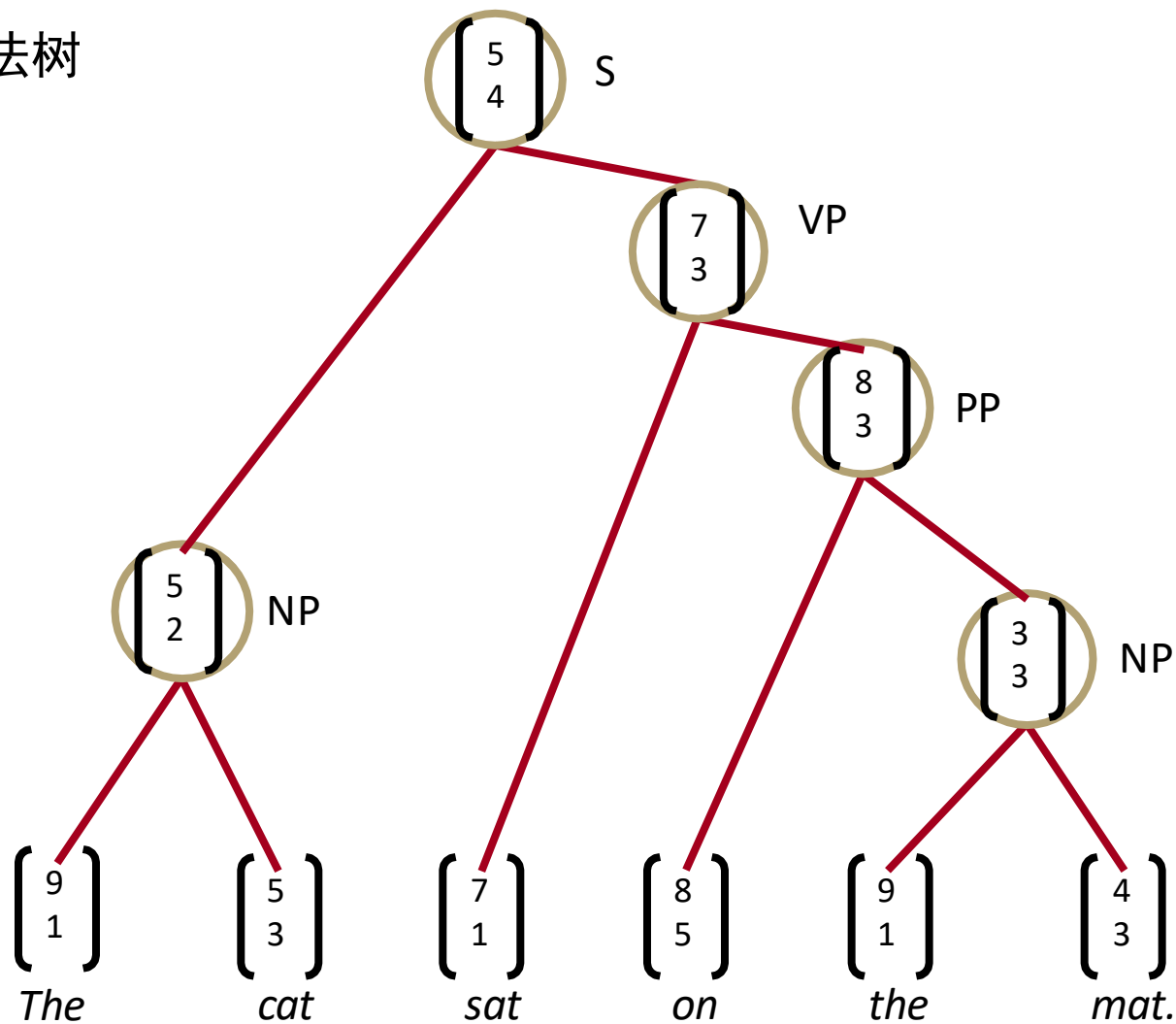
递归神经网络

- 示例
 - 给定一个语法树



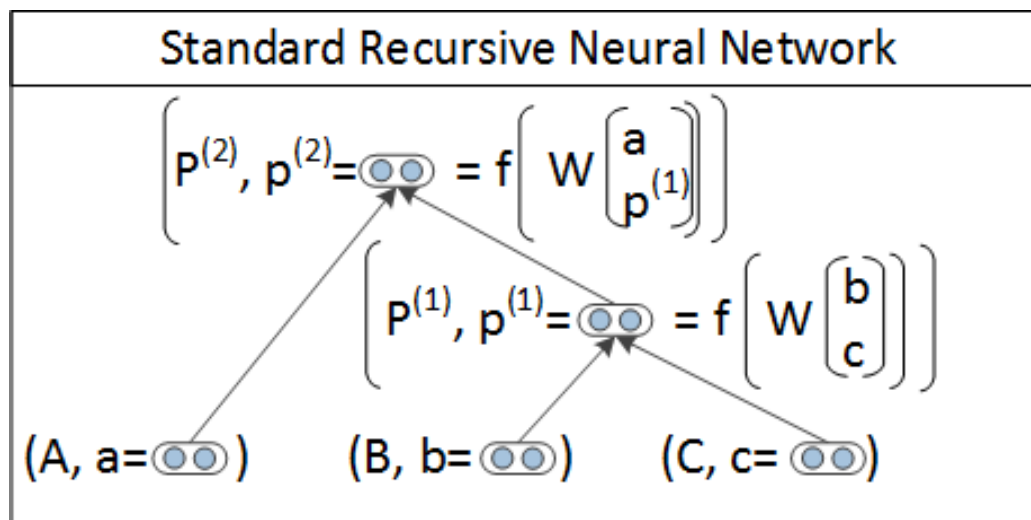
递归神经网络

- 示例
 - 给定一个语法树



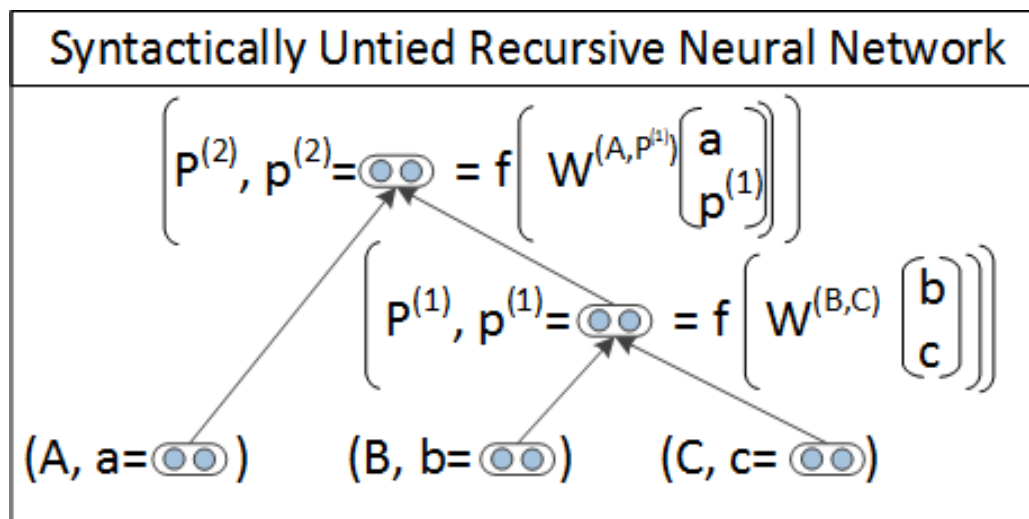
递归神经网络

- 标准递归神经网络



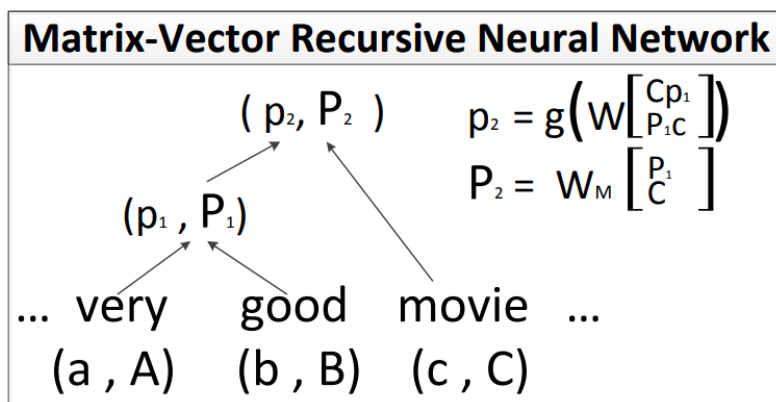
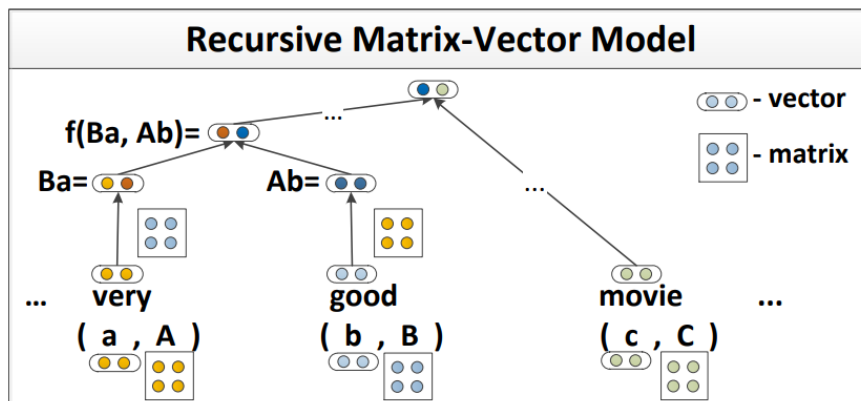
递归神经网络

- Syntactically-Untied RvNN
 - 不同的句法成分类别对应不同的聚合函数



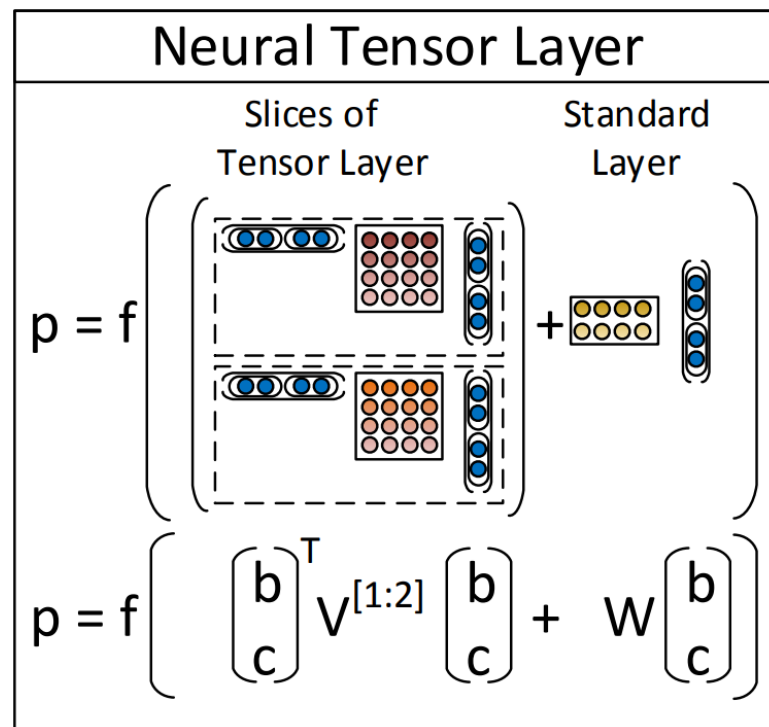
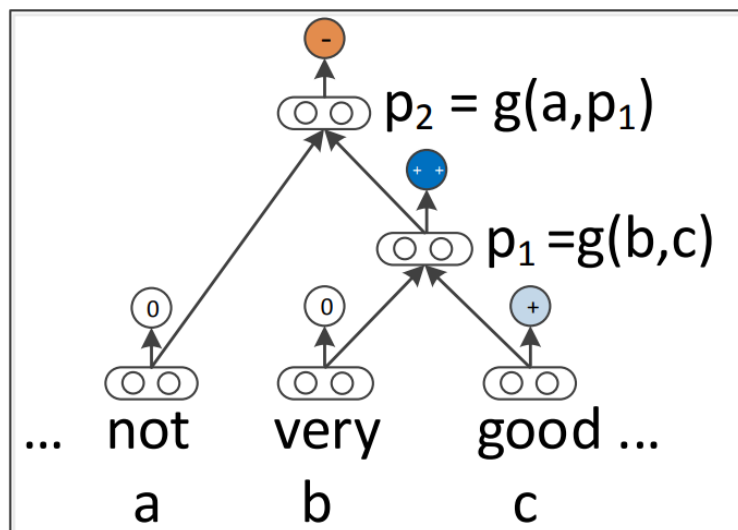
递归神经网络

- Matrix-Vector Recursive Neural Network (Richard Socher et al, 2012)
 - 每个节点由一个向量和一个矩阵来共同表示



递归神经网络

- Recursive Neural Tensor Network (Richard Socher et al, 2013)
 - 聚合时引入张量运算



递归神经网络

- Stanford Sentiment TreeBank (SST)
 - 实验结果

Model	Fine-grained		Positive/Negative	
	All	Root	All	Root
NB	67.2	41.0	82.6	81.8
SVM	64.3	40.7	84.6	79.4
BiNB	71.0	41.9	82.7	83.1
VecAvg	73.3	32.7	85.1	80.1
RNN	79.0	43.2	86.1	82.4
MV-RNN	78.7	44.4	86.8	82.9
RNTN	80.7	45.7	87.6	85.4

Table 1: Accuracy for fine grained (5-class) and binary predictions at the sentence level (root) and for all nodes.

注意力机制

- 引言
- 循环神经网络
- 参数学习
- 长程依赖问题
- 基于门控的循环神经网络
- 深度循环神经网络
- 递归神经网络
- **注意力机制**
- 循环神经网络以及注意力机制的应用

注意力机制

- 引言
 - 阅读理解

Facebook bAbi tasks

- 1 John moved to the bedroom.
- 2 Mary grabbed the football there.
- 3 Sandra journeyed to the bedroom.
- 4 Sandra went back to the hallway.
- 5 Mary moved to the garden.
- 6 Mary journeyed to the office.

Where is the football?

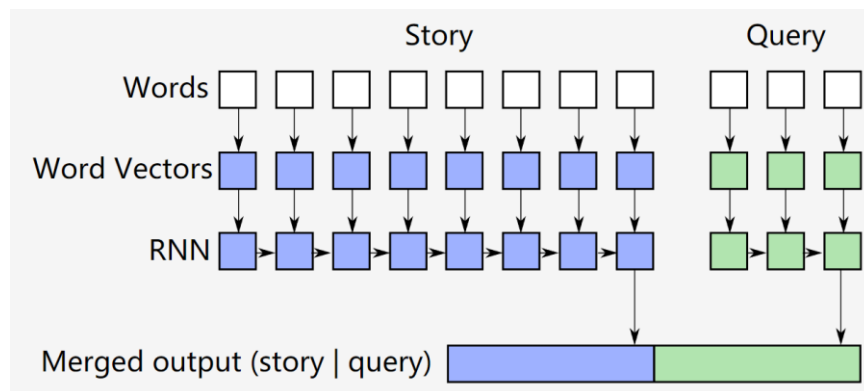
注意力机制

- 引言
 - 阅读理解

Facebook bAbi tasks

- 1 John moved to the bedroom.
- 2 Mary grabbed the football there.
- 3 Sandra journeyed to the bedroom.
- 4 Sandra went back to the hallway.
- 5 Mary moved to the garden.
- 6 Mary journeyed to the office.

Where is the football? office



注意力机制

- 引言

- 机器阅读理解

- Stanford Question Answering Dataset (SQuAD)

The first recorded travels by Europeans to China and back date from this time. The most famous traveler of the period was the Venetian Marco Polo, whose account of his trip to "Cambaluc," the capital of the Great Khan, and of life there astounded the people of Europe. The account of his travels, *Il milione* (or, *The Million*, known in English as the *Travels of Marco Polo*), appeared about the year 1299. Some argue over the accuracy of Marco Polo's accounts due to the lack of mentioning the Great Wall of China, tea houses, which would have been a prominent sight since Europeans had yet to adopt a tea culture, as well the practice of foot binding by the women in capital of the Great Khan. Some suggest that Marco Polo acquired much of his knowledge through contact with Persian traders since many of the places he named were in Persian.

How did some suspect that Polo learned about China instead of by actually visiting it?

Answer: through contact with Persian traders

注意力机制

- 引言

- 大脑中的注意力

- 人脑每个时刻接收的外界输入信息非常多，包括来源于视觉、听觉、触觉的各种各样的信息。
 - 但就视觉来说，眼睛每秒钟都会发送千万比特的信息给视觉神经系统。
 - 人脑通过**注意力**来解决**信息超载问题**。

注意力机制

- 引言
 - 注意力示例



注意力机制

- 引言

- 注意力示例

- 当一个人在吵闹的鸡尾酒会上和朋友聊天时，尽管周围噪音干扰很多，他还是可以听到朋友的谈话内容，而忽略其他人的声音。
 - 同时，如果未注意到的背景声中有重要的词（比如他的名字），他会马上注意到。

注意力机制

- 引言
 - 注意力实验



所有黑色牌的数字之和为多少？

注意力机制

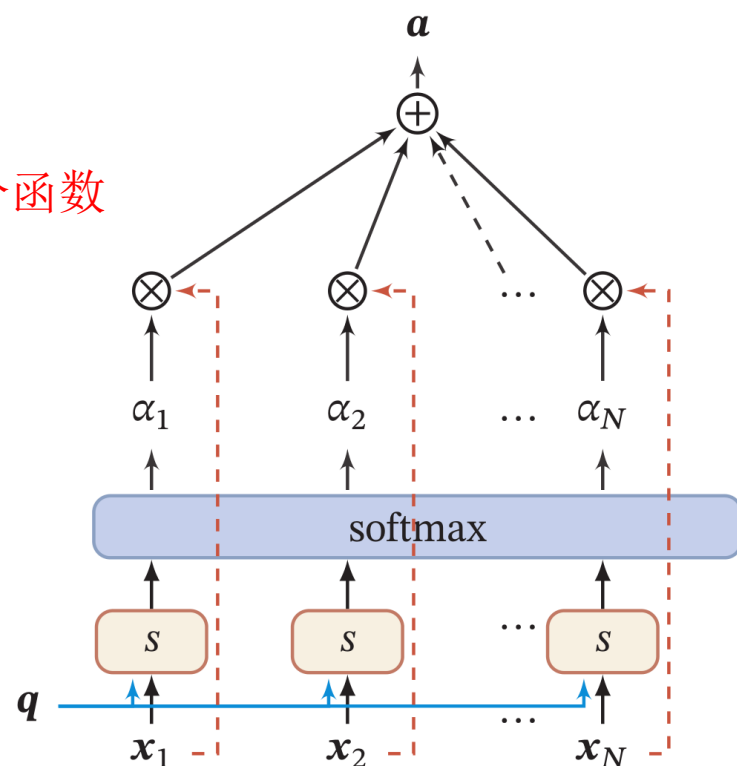
- 注意力模型

- 软性注意力机制 (soft attention mechanism)
- 注意力机制可以分为两步
 - 计算注意力分布 α ,

$$\begin{aligned}\alpha_n &= p(z = n | X, q) \\ &= \text{softmax}(s(x_n, q)) \quad s(x_n, q) \text{ 打分函数} \\ &= \frac{\exp(s(x_n, q))}{\sum_{j=1}^N \exp(s(x_j, q))}\end{aligned}$$

- 根据 α 来计算输入信息的加权平均。

$$\begin{aligned}\text{att}(X, q) &= \sum_{n=1}^N \alpha_n x_n \\ &= \mathbb{E}_{z \sim p(z|X, q)}[x_z]\end{aligned}$$



注意力机制

- 注意力模型
 - 注意力打分函数 $s(x_n, q)$

加性模型

$$s(\mathbf{x}, \mathbf{q}) = \mathbf{q}^T \tanh(\mathbf{W}\mathbf{x} + \mathbf{b})$$

点积模型

$$s(\mathbf{x}, \mathbf{q}) = \mathbf{x}^T \mathbf{q}$$

缩放点积模型

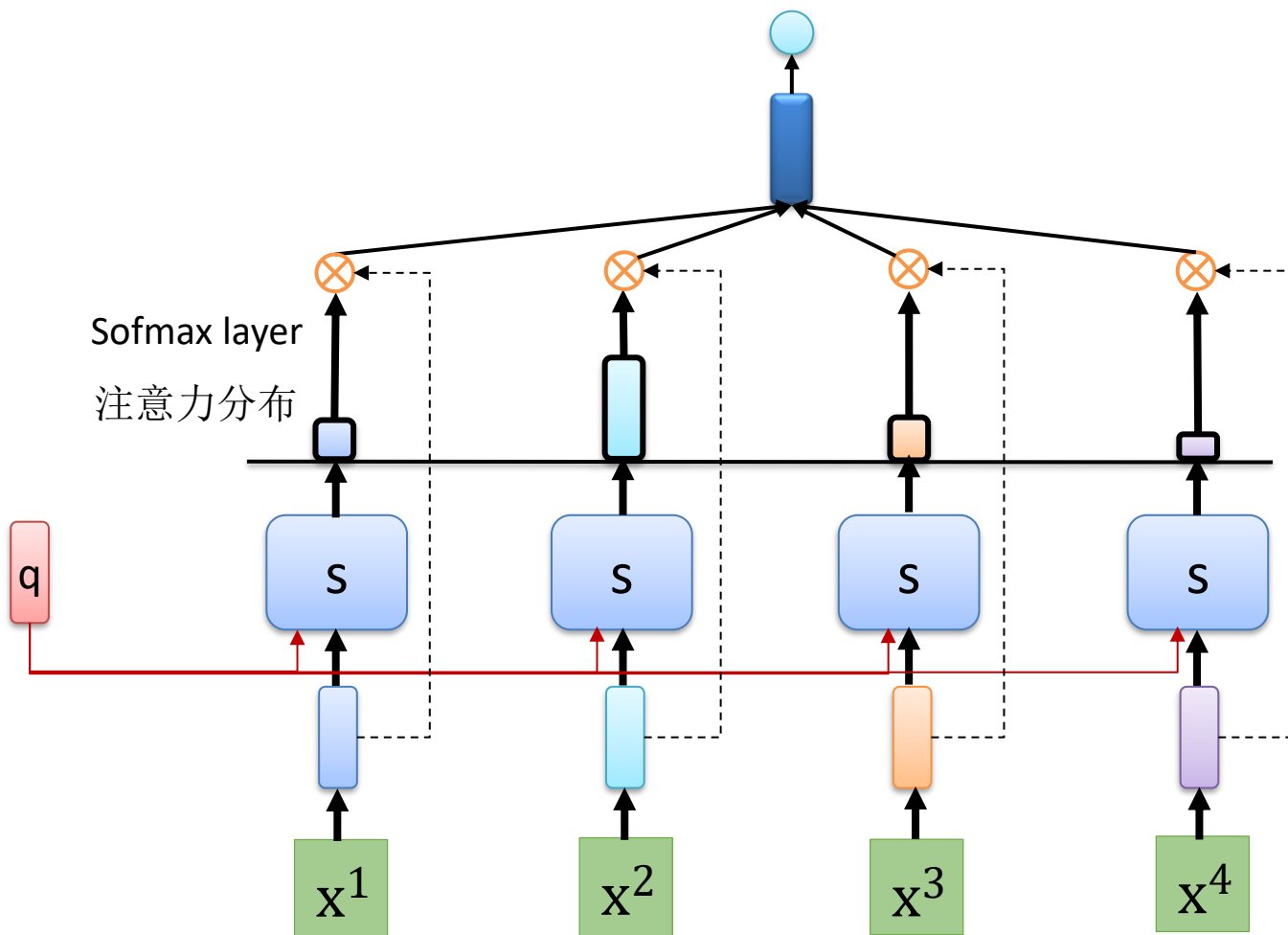
$$s(\mathbf{x}, \mathbf{q}) = \frac{\mathbf{x}^T \mathbf{q}}{\sqrt{D}}$$

双线性模型

$$s(\mathbf{x}, \mathbf{q}) = \mathbf{x}^T \mathbf{W} \mathbf{q}$$

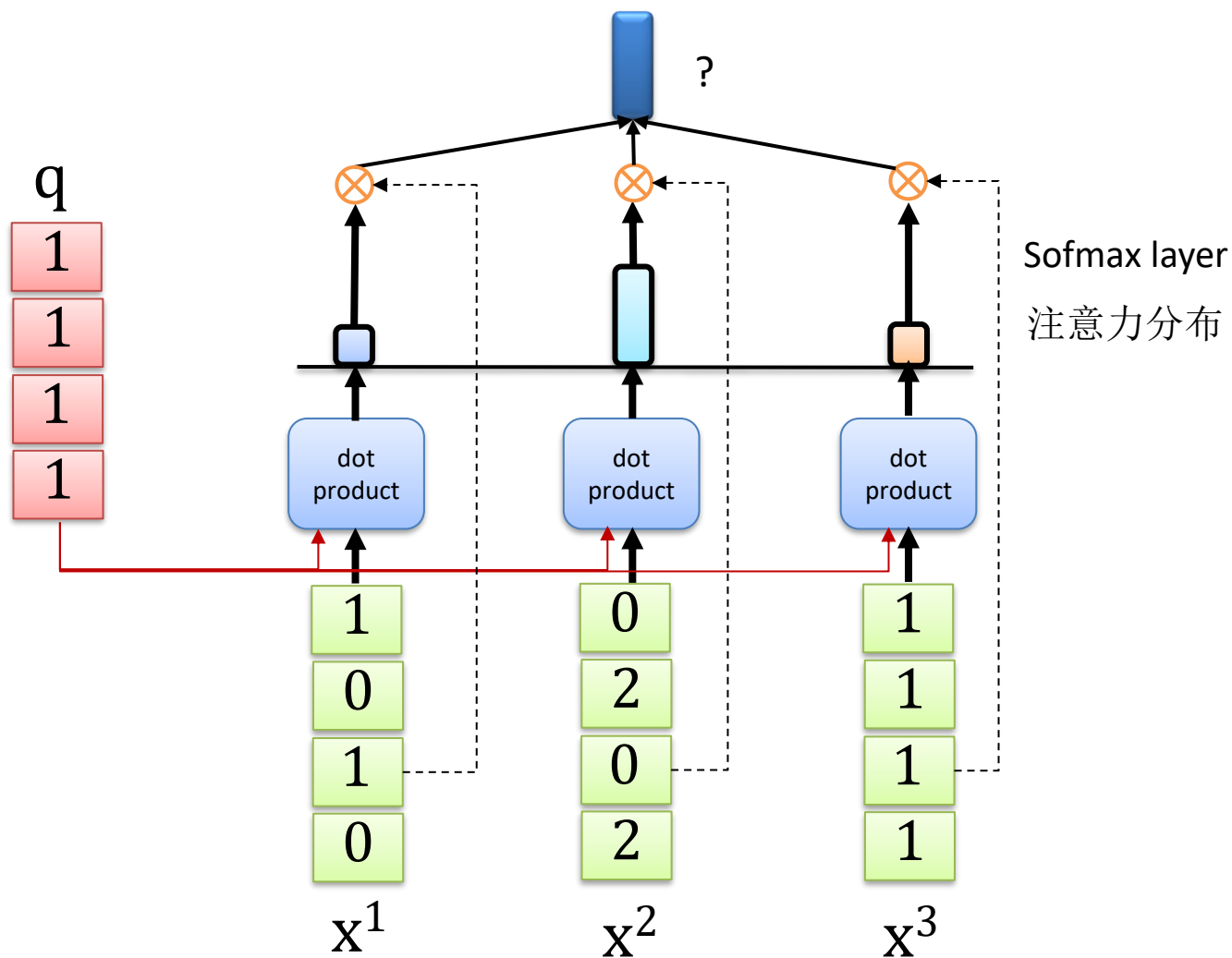
注意力机制

- 注意力机制举例



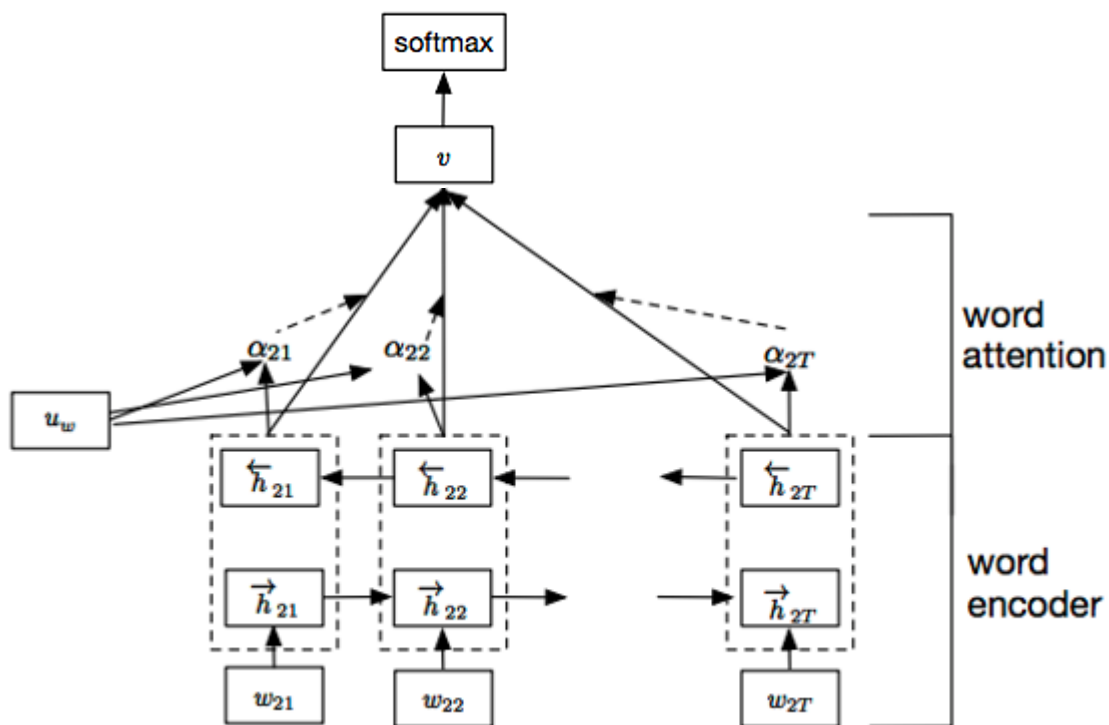
注意力机制

- 注意力机制举例
 - 课堂练习



注意力机制

- 常见的注意力模型
 - 句子分类



注意力机制

- 常见的注意力模型
 - 文档分类

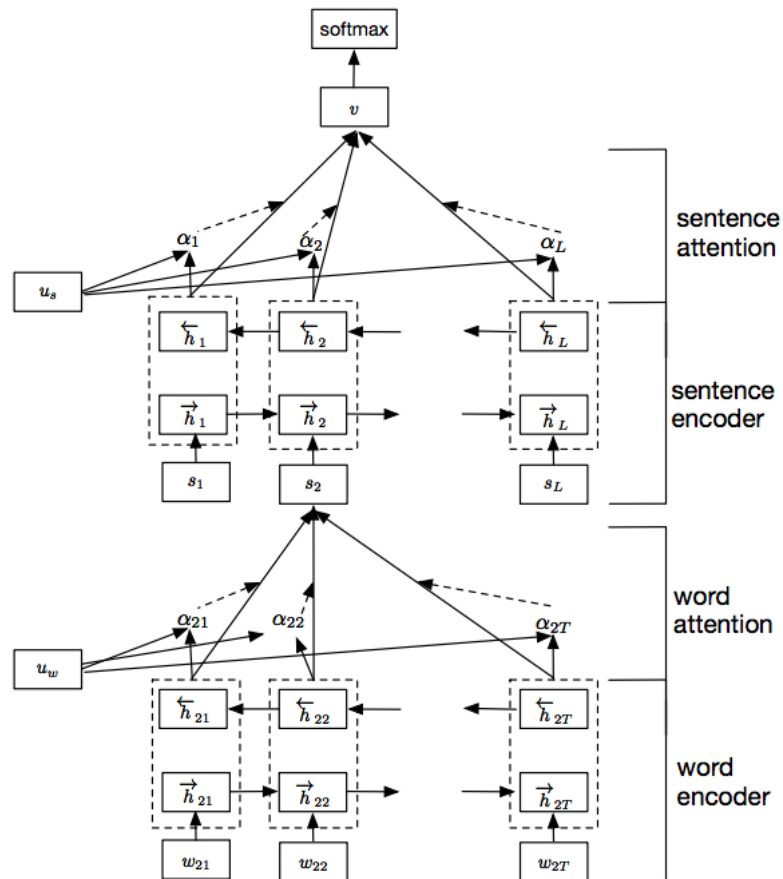
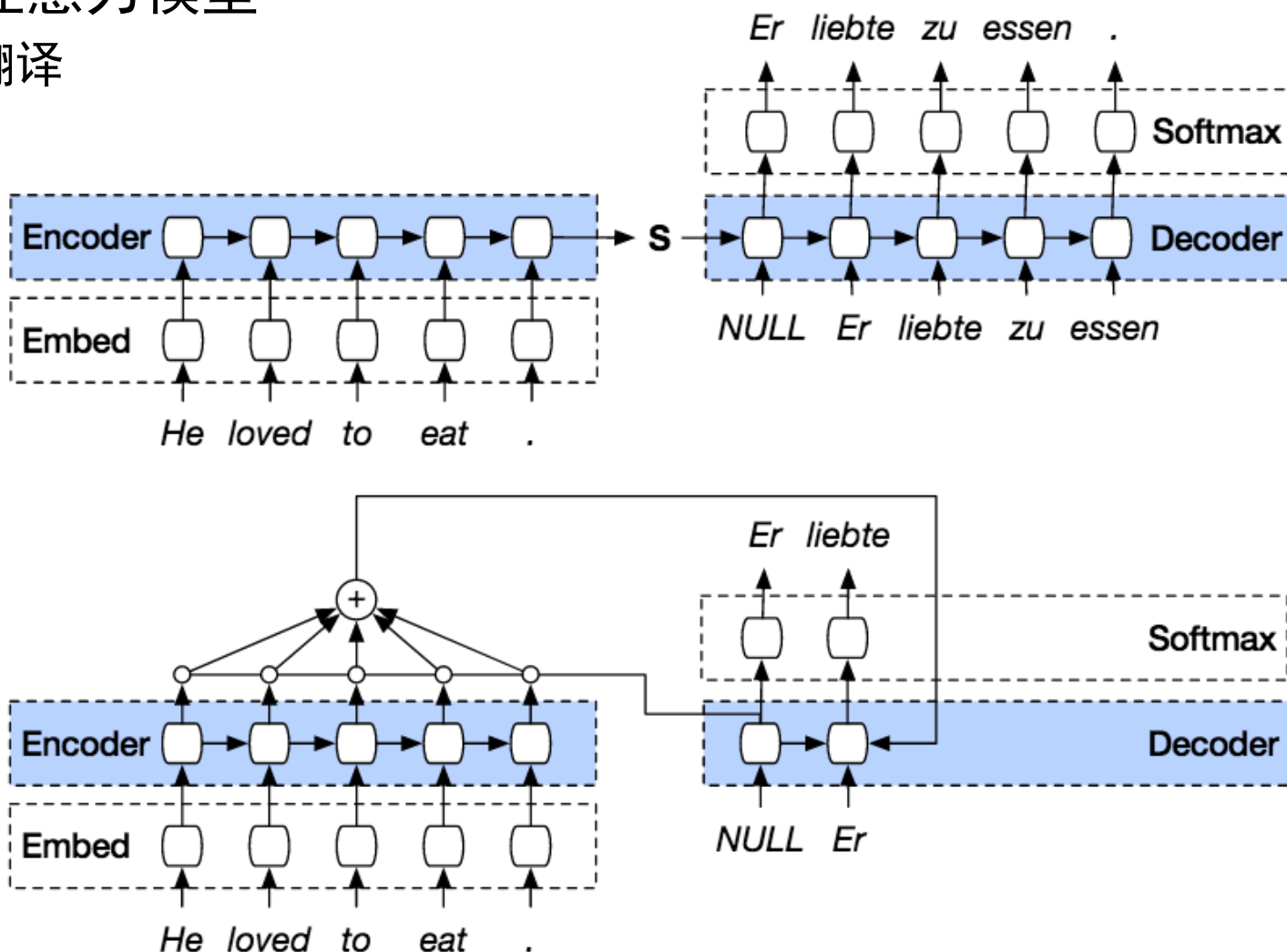


Figure 2: Hierarchical Attention Network.

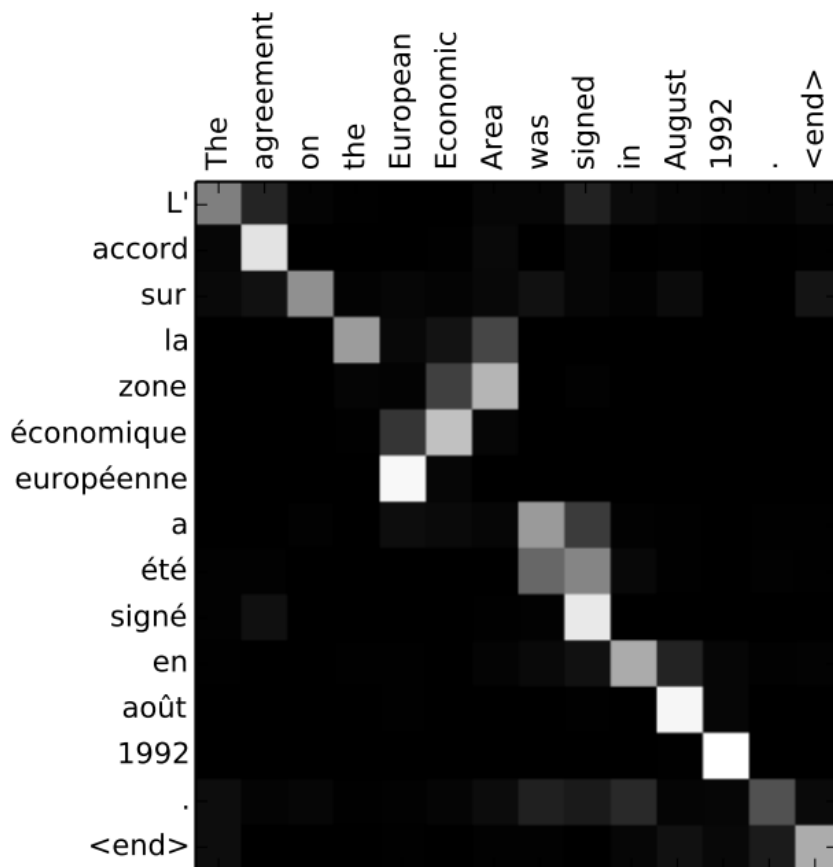
注意力机制

- 常见的注意力模型
 - 机器翻译



注意力机制

- 常见的注意力模型
 - 机器翻译



注意力机制

- Hierarchical Attention Network
 - 代码实现

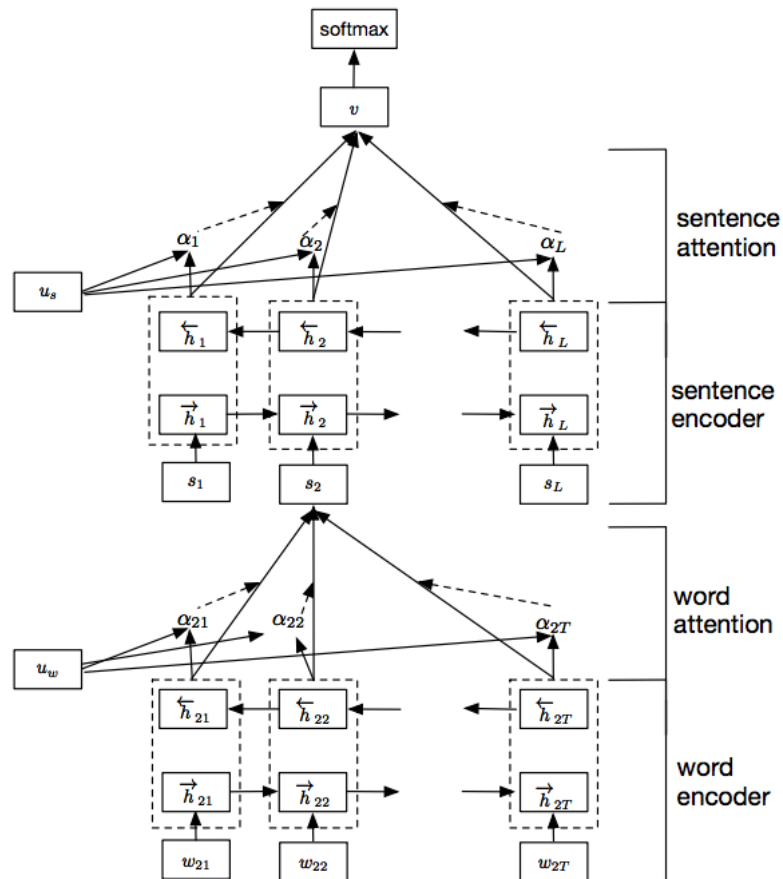


Figure 2: Hierarchical Attention Network.

注意力机制

- Hierarchical Attention Network
 - 代码实现

```
9 class WordAttention(nn.Module):
10
11     def __init__(self, vocab_size, emb_size, word_rnn_size, word_rnn_layers, word_att_size, dropout):
12
13         super(WordAttention, self).__init__()
14
15         # Embeddings (look-up) layer
16         self.embeddings = nn.Embedding(vocab_size, emb_size)
17
18         # Bidirectional word-level RNN
19         self.word_rnn = nn.GRU(emb_size, word_rnn_size, num_layers=word_rnn_layers, bidirectional=True,
20                                dropout=dropout, batch_first=True)
21
22         # Word-level attention network
23         self.word_attention = nn.Linear(2 * word_rnn_size, word_att_size)
24
25         # Word context vector to take dot-product with
26         self.word_context_vector = nn.Linear(word_att_size, 1, bias=False)
27
28         self.dropout = nn.Dropout(dropout)
29
30     def forward(self, sentences, words_per_sentence):
31         # Sort sentences by decreasing sentence lengths (SORTING #2)
32         words_per_sentence, sent_sort_ind = words_per_sentence.sort(dim=0, descending=True)
33         sentences = sentences[sent_sort_ind] # (n_sentences, word_pad_len, emb_size)
34
35         # Get word embeddings, apply dropout
36         sentences = self.dropout(self.embeddings(sentences)) # (n_sentences, word_pad_len, emb_size)
```

```
57
58     # Re-arrange as words by removing pad-words (SENTENCES -> WORDS)
59     words, bw = pack_padded_sequence(sentences,
60                                     lengths=words_per_sentence.tolist(),
61                                     batch_first=True)
62     # (n_words, emb_size), bw is the effective batch size at each word-timestep
63
64     (words, _), _ = self.word_rnn(PackedSequence(words, bw)) # (n_words, 2 * word_rnn_size), (max(sent_lens))
65
66     # Find attention vectors by applying the attention linear layer
67     att_w = self.word_attention(words) # (n_words, att_size)
68     att_w = F.tanh(att_w) # (n_words, att_size)
69
70     # Take the dot-product of the attention vectors with the context vector (i.e. parameter of linear layer)
71     att_w = self.word_context_vector(att_w).squeeze(1) # (n_words)
72
73     max_value = att_w.max()
74     att_w = torch.exp(att_w - max_value) # (n_words)
75
76     # Re-arrange as sentences by re-padding with 0s (WORDS -> SENTENCES)
77     att_w, _ = pad_packed_sequence(PackedSequence(att_w, bw), batch_first=True)
78     # (n_sentences, max_sent_len_in_batch)
79
80     # Calculate softmax values
81     word_alphas = att_w / torch.sum(att_w, dim=1, keepdim=True)
82
83     # (n_sentences, max_sent_len_in_batch)
84
85     # Similarly re-arrange word-level RNN outputs as sentences by re-padding with 0s (WORDS -> SENTENCES)
86     sentences, _ = pad_packed_sequence(PackedSequence(words, bw), batch_first=True)
87     # (n_sentences, max_sent_len_in_batch, 2 * word_rnn_size)
88
89     # Find sentence embeddings
90     sentences = sentences * word_alphas.unsqueeze(2) # (n_sentences, max_sent_len_in_batch, 2 * word_rnn_size)
91     sentences = sentences.sum(dim=1) # (n_sentences, 2 * word_rnn_size)
92
93     # Unsort sentences into the original order (INVERSE OF SORTING #2)
94     _, sent_unsort_ind = sent_sort_ind.sort(dim=0, descending=False) # (n_sentences)
95     sentences = sentences[sent_unsort_ind] # (n_sentences, 2 * word_rnn_size)
96     word_alphas = word_alphas[sent_unsort_ind] # (n_sentences, max_sent_len_in_batch)
97
98     return sentences, word_alphas
```

注意力机制

- Hierarchical Attention Network
 - 代码实现

```
86 class SentenceAttention(nn.Module):
87     def __init__(self, vocab_size, emb_size, word_rnn_size, sentence_rnn_size, word_rnn_layers, sentence_rnn_layers,
88                 word_att_size, sentence_att_size, dropout):
89         super(SentenceAttention, self).__init__()
90
91         # Word-level attention module
92         self.word_attention = WordAttention(vocab_size, emb_size, word_rnn_size, word_rnn_layers, word_att_size,
93                                           dropout)
94
95         # Bidirectional sentence-level RNN
96         self.sentence_rnn = nn.GRU(2 * word_rnn_size, sentence_rnn_size, num_layers=sentence_rnn_layers,
97                                   bidirectional=True, dropout=dropout, batch_first=True)
98
99         # Sentence-level attention network
100        self.sentence_attention = nn.Linear(2 * sentence_rnn_size, sentence_att_size)
101
102        # Sentence context vector to take dot-product with
103        self.sentence_context_vector = nn.Linear(sentence_att_size, 1, bias=False)
104        # this performs a dot product with the linear layer's 1D parameter vector, which is the sentence context vector
105
106        # You could also do this with:
107        # self.sentence_context_vector = nn.Parameter(torch.FloatTensor(1, sentence_att_size))
108        # self.sentence_context_vector.data.uniform_(-0.1, 0.1)
109        # And then take the dot-product
110
111        # Dropout
112        self.dropout = nn.Dropout(dropout)
```

```
114 def forward(self, documents, sentences_per_document, words_per_sentence):
115
116     # Sort documents by decreasing document lengths (SORTING #1)
117     sentences_per_document, doc_sort_ind = sentences_per_document.sort(dim=0, descending=True)
118     documents = documents[doc_sort_ind] # (n_documents, sent_pad_len, word_pad_len)
119     words_per_sentence = words_per_sentence[doc_sort_ind] # (n_documents, sent_pad_len)
120
121     # Re-arrange as sentences by removing pad-sentences (DOCUMENTS -> SENTENCES)
122     sentences, bs = pack_padded_sequence(documents,
123                                         lengths=sentences_per_document.tolist(),
124                                         batch_first=True)
125     # (n_sentences, word_pad_len), bs is the effective batch size at each sentence-timestep
```

```
127     # Re-arrange sentence lengths in the same way (DOCUMENTS -> SENTENCES)
128     words_per_sentence, _ = pack_padded_sequence(words_per_sentence,
129                                                  lengths=sentences_per_document.tolist(),
130                                                  batch_first=True)
131     # (n_sentences), '_' is the same as 'bs' in the earlier step
132
133     # Find sentence embeddings by applying the word-level attention module
134     sentences, word_alphas = self.word_attention(sentences, words_per_sentence)
135
136     # (n_sentences, 2 * word_rnn_size), (n_sentences, max_sent_len_in_batch)
137     sentences = self.dropout(sentences)
138
139     (sentences, _), _ = self.sentence_rnn(
140         PackedSequence(sentences, bs)) # (n_sentences, 2 * sentence_rnn_size), (max(sent_lens))
141
142     # Find attention vectors by applying the attention linear layer
143     att_s = self.sentence_attention(sentences) # (n_sentences, att_size)
144     att_s = F.tanh(att_s) # (n_sentences, att_size)
145
146     # Take the dot-product of the attention vectors with the context vector (i.e. parameter of linear layer)
147     att_s = self.sentence_context_vector(att_s).squeeze(1) # (n_sentences)
148
149     max_value = att_s.max() # scalar, for numerical stability during exponent calculation
150     att_s = torch.exp(att_s - max_value) # (n_sentences)
151
152     # Re-arrange as documents by re-padding with 0s (SENTENCES -> DOCUMENTS)
153     att_s, _ = pad_packed_sequence(PackedSequence(att_s, bs),
154                                   batch_first=True) # (n_documents, max_doc_len_in_batch)
155
156     # Calculate softmax values
157     sentence_alphas = att_s / torch.sum(att_s, dim=1, keepdim=True) # (n_documents, max_doc_len_in_batch)
158
159     # Similarly re-arrange sentence-level RNN outputs as documents by re-padding with 0s (SENTENCES -> DOCUMENTS)
160     documents, _ = pad_packed_sequence(PackedSequence(sentences, bs),
161                                       batch_first=True)
162     # (n_documents, max_doc_len_in_batch, 2 * sentence_rnn_size)
163
164     # Find document embeddings
165     documents = documents * sentence_alphas.unsqueeze(
166         2) # (n_documents, max_doc_len_in_batch, 2 * sentence_rnn_size)
167     documents = documents.sum(dim=1) # (n_documents, 2 * sentence_rnn_size)
168
169     # Also re-arrange word_alphas (SENTENCES -> DOCUMENTS)
170     word_alphas, _ = pad_packed_sequence(PackedSequence(word_alphas, bs),
171                                         batch_first=True)
172     # (n_documents, max_doc_len_in_batch, max_sent_len_in_batch)
173
174     # Unsort documents into the original order (INVERSE OF SORTING #1)
175     _, doc_unsort_ind = doc_sort_ind.sort(dim=0, descending=False) # (n_documents)
176     documents = documents[doc_unsort_ind] # (n_documents, 2 * sentence_rnn_size)
177     sentence_alphas = sentence_alphas[doc_unsort_ind] # (n_documents, max_doc_len_in_batch)
178     word_alphas = word_alphas[doc_unsort_ind] # (n_documents, max_doc_len_in_batch, max_sent_len_in_batch)
179
180     return documents, word_alphas, sentence_alphas
```

注意力机制

- Hierarchical Attention Network
 - 代码实现

```
183 class HierarchicalAttentionNetwork(nn.Module):
184
185     def __init__(self, n_classes, vocab_size, emb_size, word_rnn_size, sentence_rnn_size, word_rnn_layers,
186                 sentence_rnn_layers, word_att_size, sentence_att_size, dropout=0.5):
187
188         super(HierarchicalAttentionNetwork, self).__init__()
189
190         self.sentence_attention = SentenceAttention(vocab_size, emb_size, word_rnn_size, sentence_rnn_size,
191                                                    word_rnn_layers, sentence_rnn_layers, word_att_size,
192                                                    sentence_att_size, dropout)
193
194         # Classifier
195         self.fc = nn.Linear(2 * sentence_rnn_size, n_classes)
196
197         self.dropout = nn.Dropout(dropout)
198
199     def forward(self, documents, sentences_per_document, words_per_sentence):|
200
201         # (n_documents, 2 * sentence_rnn_size)
202         # (n_documents, max_doc_len_in_batch, max_sent_len_in_batch)
203         # (n_documents, max_doc_len_in_batch)
204         document_embeddings, word_alphas, sentence_alphas = self.sentence_attention(documents, sentences_per_document,
205                                                                                     words_per_sentence)
206
207         # (n_documents, n_classes)
208         scores = self.fc(self.dropout(document_embeddings))
209
210         return scores, word_alphas, sentence_alphas
```

循环神经网络的应用

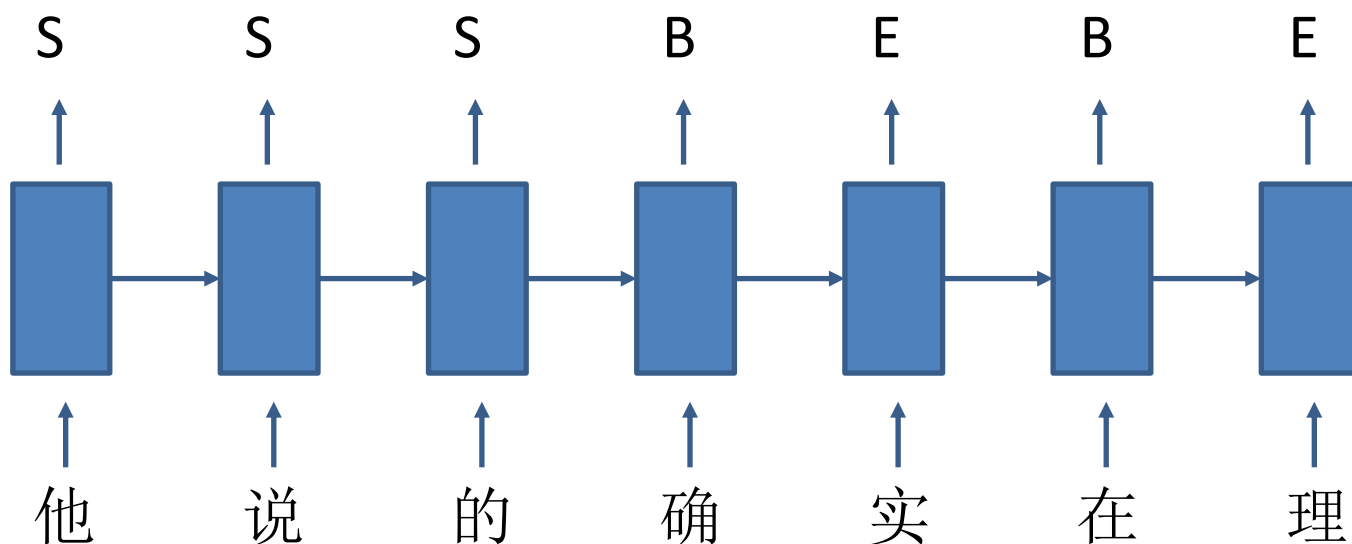
- 引言
- 循环神经网络
- 参数学习
- 长程依赖问题
- 基于门控的循环神经网络
- 深度循环神经网络
- 递归神经网络
- 注意力机制
- 循环神经网络以及注意力机制的应用
 - 序列标注模型
 - 机器阅读理解
 - 自然语言推断、文本生成、看图说话等

循环神经网络的应用

- 引言
- 循环神经网络
- 参数学习
- 长程依赖问题
- 基于门控的循环神经网络
- 深度循环神经网络
- 递归神经网络
- 注意力机制
- 循环神经网络以及注意力机制的应用
 - 序列标注任务
 - 机器阅读理解
 - 自然语言推断、文本生成、看图说话等

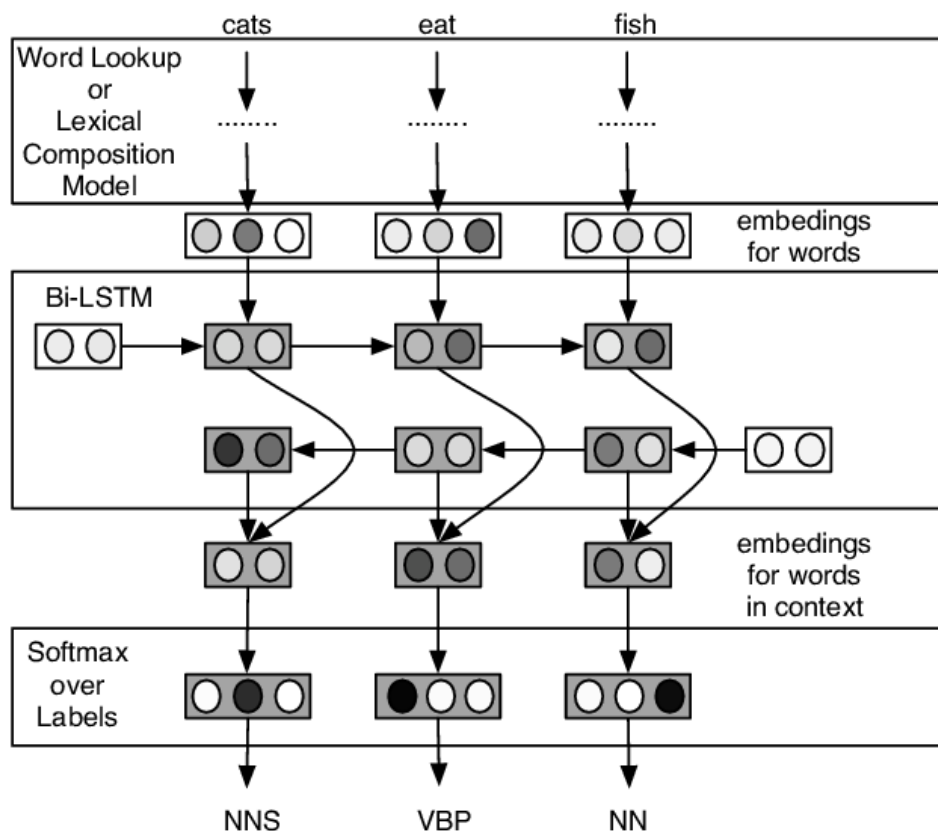
序列标注任务

- 中文分词 (Chinese Word Segmentation, CWS)
 - 中文分词就是将连续的字序列按照一定的规范重新组合成语义独立词序列的过程。



序列标注任务

- 词性标注 (Part-Of-Speech Tagging, POS Tagging)
 - 指为分词结果中的每个单词标注一个正确的词性的程序，也即确定每个词是名词、动词、形容词或者其他词性的过程。



Number	Tag	Description
1.	CC	Coordinating conjunction
2.	CD	Cardinal number
3.	DT	Determiner
4.	EX	Existential <i>there</i>
5.	FW	Foreign word
6.	IN	Preposition or subordinating conjunction
7.	JJ	Adjective
8.	JJR	Adjective, comparative
9.	JJS	Adjective, superlative
10.	LS	List item marker
11.	MD	Modal
12.	NN	Noun, singular or mass
13.	NNS	Noun, plural
14.	NNP	Proper noun, singular
15.	NNPS	Proper noun, plural
16.	PDT	Predeterminer
17.	POS	Possessive ending
18.	PRP	Personal pronoun
19.	PRP\$	Possessive pronoun
20.	RB	Adverb
21.	RBR	Adverb, comparative
22.	RBS	Adverb, superlative
23.	RP	Particle
24.	SYM	Symbol
25.	TO	<i>to</i>
26.	UH	Interjection
27.	VB	Verb, base form
28.	VBD	Verb, past tense
29.	VBG	Verb, gerund or present participle
30.	VBN	Verb, past participle
31.	VBP	Verb, non-3rd person singular present
32.	VBZ	Verb, 3rd person singular present
33.	WDT	Wh-determiner
34.	WP	Wh-pronoun
35.	WPS	Possessive wh-pronoun
36.	WRB	Wh-adverb

序列标注任务

- 命名实体识别 (Named Entity Recognition, NER)
 - 识别出待处理文本中提及的命名实体，主要包括人名(PER)、机构名(ORG)、地名(LOC)、时间、日期和货币等七类实体
 - 人名（例如“乔布斯”）、地名（例如“北京”）、组织机构名（例如“中国科学院”）、时间（例如“10点30分”）、日期（例如“2017年6月1日”）、货币（例如“1000美元”）、百分比（例如“百分之五十”）
 - 由于时间、日期、货币和百分比规则性强，利用模板或正则表达式基本可处理
 - 人名、地名和组织机构名是关注重点
 - 大部分现有数据集主要识别四类实体，包括人名(PER)、机构名(ORG)、地名(LOC)以及其他专有名词 (Miscellaneous)

序列标注任务

命名实体识别

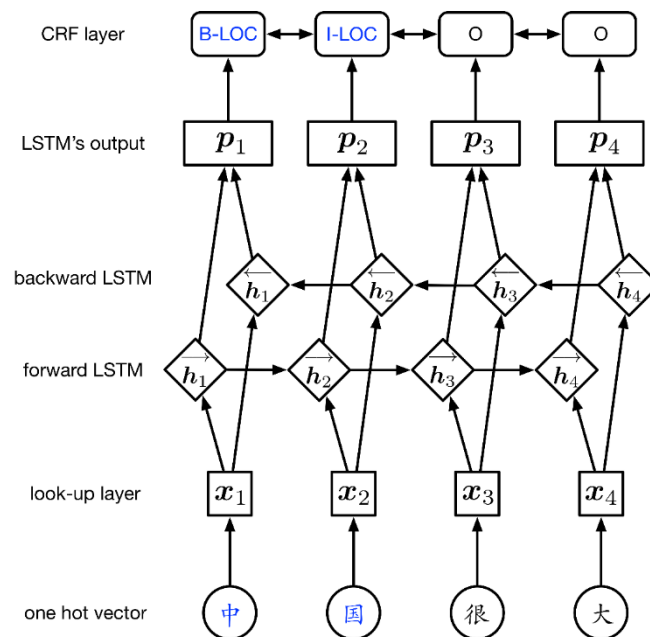
包含两个任务：实体检测和实体分类

- 实体检测：检测出文本中哪些词串属于实体，也即发现实体的左边界和右边界
- 实体分类：判别检测出的实体具体属于哪个类别

中国/LOC 很大。
陈明亮/PER 是一所小学的校长。
北京市发改委/ORG 出台了政策。
国科大/ORG 是最近新成立的一所大学。

转换为基于字的序列标注问题

中/B-LOC 国/I-LOC 很/O 大/O。
陈/B-PER 明/I-PER 亮/I-PER 是/O 一/O 所/O 小/O 学/O 的/O 校/O 长/O。
北/B-ORG 京/I-ORG 市/I-ORG 发/I-ORG 改/I-ORG 委/I-ORG 出/O 台/O 了/O 政/O 策/O。
国/B-ORG 科/I-ORG 大/I-ORG 是/O 最/O 近/O 新/O 成/O 立/O 的/O 一/O 所/O 大/O 学/O。



序列标注任务

- NER评估指标

- Precision, Recall, F1
- True Positive (TP): D 和 $D+$ 中标注完全一致的命名实体数目
- False Positive (FP): D 中系统识别出但参考答案 $D+$ 中没有的命名实体数目
- False Negative (FN): $D+$ 参考答案中存在但 D 中系统未识别出的命名实体数目

$$\text{Precision} = \frac{TP}{TP+FP}, \text{Recall} = \frac{TP}{TP+FN}, \text{F1} = \frac{2PR}{P+R}$$

序列标注任务

- NER评估指标
 - Precision, Recall, F1

Words	True Label	Predicted Label
Foreign	ORG	O
Ministry	ORG	O
spokesman	O	O
Shen	PER	PER
Guofang	PER	O
told	O	O
Reuters	ORG	ORG

- TP, FP, FN = ? 1 1 2
- Precision, Recall, F1 = ?

序列标注任务

- NER评估指标
 - Precision, Recall, F1

	Words	True Label	Predicted Label	
entity	Foreign	ORG	O	FN
	Ministry	ORG	O	
	spokesman	O	O	
entity	Shen	PER	PER	FP
	Guofang	PER	O	
	told	O	O	
entity	Reuters	ORG	ORG	TP

循环神经网络的应用

- 引言
- 循环神经网络
- 参数学习
- 长程依赖问题
- 基于门控的循环神经网络
- 深度循环神经网络
- 递归神经网络
- 注意力机制
- 循环神经网络以及注意力机制的应用
 - 序列标注任务
 - 机器阅读理解
 - 自然语言推断、文本生成、看图说话等

机器阅读理解

- Stanford Question Answering Dataset (SQuAD, Rajpurkar et al., 2016)
 - 包含10万个（问题，原文，答案）三元组
 - 让标注人员提出最多5个基于文章内容的问题并提供正确答案，且答案出现在原文中
 - 也叫作抽取式问答（Extractive Question Answering）

Question: Which team won Super Bowl 50?

Passage

Super Bowl 50 was an American football game to determine the champion of the National Football League (NFL) for the 2015 season. The American Football Conference (AFC) champion **Denver Broncos** defeated the National Football Conference (NFC) champion Carolina Panthers 24–10 to earn their third Super Bowl title. The game was played on February 7, 2016, at Levi's Stadium in the San Francisco Bay Area at Santa Clara, California.

机器阅读理解

- Stanford Question Answering Dataset (SQuAD)
 - 验证集和测试集每个问题含有3个Ground-Truth答案

Private schools, also known as independent schools, non-governmental, or nonstate schools, are not administered by local, state or national governments; thus, they retain the right to select their students and are funded in whole or in part by charging their students tuition, rather than relying on mandatory taxation through public (government) funding; at some private schools students may be able to get a scholarship, which makes the cost cheaper, depending on a talent the student may have (e.g. sport scholarship, art scholarship, academic scholarship), financial need, or tax credit scholarships that might be available.

Along with non-governmental and nonstate schools, what is another name for private schools?

Gold answers: (1) independent (2) independent schools (3) independent schools

Along with sport and art, what is a type of talent scholarship?

Gold answers: (1) academic (2) academic (3) academic

Rather than taxation, what are private schools largely funded by?

Gold answers: (1) tuition (2) charging their students tuition (3) tuition

机器阅读理解

- Stanford Question Answering Dataset (SQuAD)

- 评估指标

- **Exact match(EM)**: 1/0 accuracy on whether you match one of the 3 answers
 - **F1**: Take system and each gold answer as bag of words, evaluate, and take the max.

$$\text{Precision} = \frac{TP}{TP+FP}, \text{Recall} = \frac{TP}{TP+FN}, \text{F1} = \frac{2PR}{P+R}$$

Along with non-governmental and nonstate schools, what is another name for private schools?

Gold answers: (1) independent (2) independent schools (3) independent schools

1) **System answers:** independent schools; **bag-of-words:** {independent, schools} Precision = 2 / 2 = 1; Recall: 2 / 2 = 1; F1 = 2 / 2 = 1

2) **System answers:** independent schools; **bag-of-words:** {independent} Precision = 1/2 = 0.5; Recall: 1 / 1 = 1; F1 = (2*0.5*1) / 1.5 = 0.67

机器阅读理解

- Stanford Question Answering Dataset (SQuAD)

- 评估指标

- Exact match(EM): 1/0 accuracy on whether you match one of the 3 answers
 - F1: Take system and each gold answer as bag of words, evaluate, and take the max.

$$\text{Precision} = \frac{TP}{TP+FP}, \text{Recall} = \frac{TP}{TP+FN}, \text{F1} = \frac{2PR}{P+R}$$

Along with non-governmental and nonstate schools, what is another name for private schools?

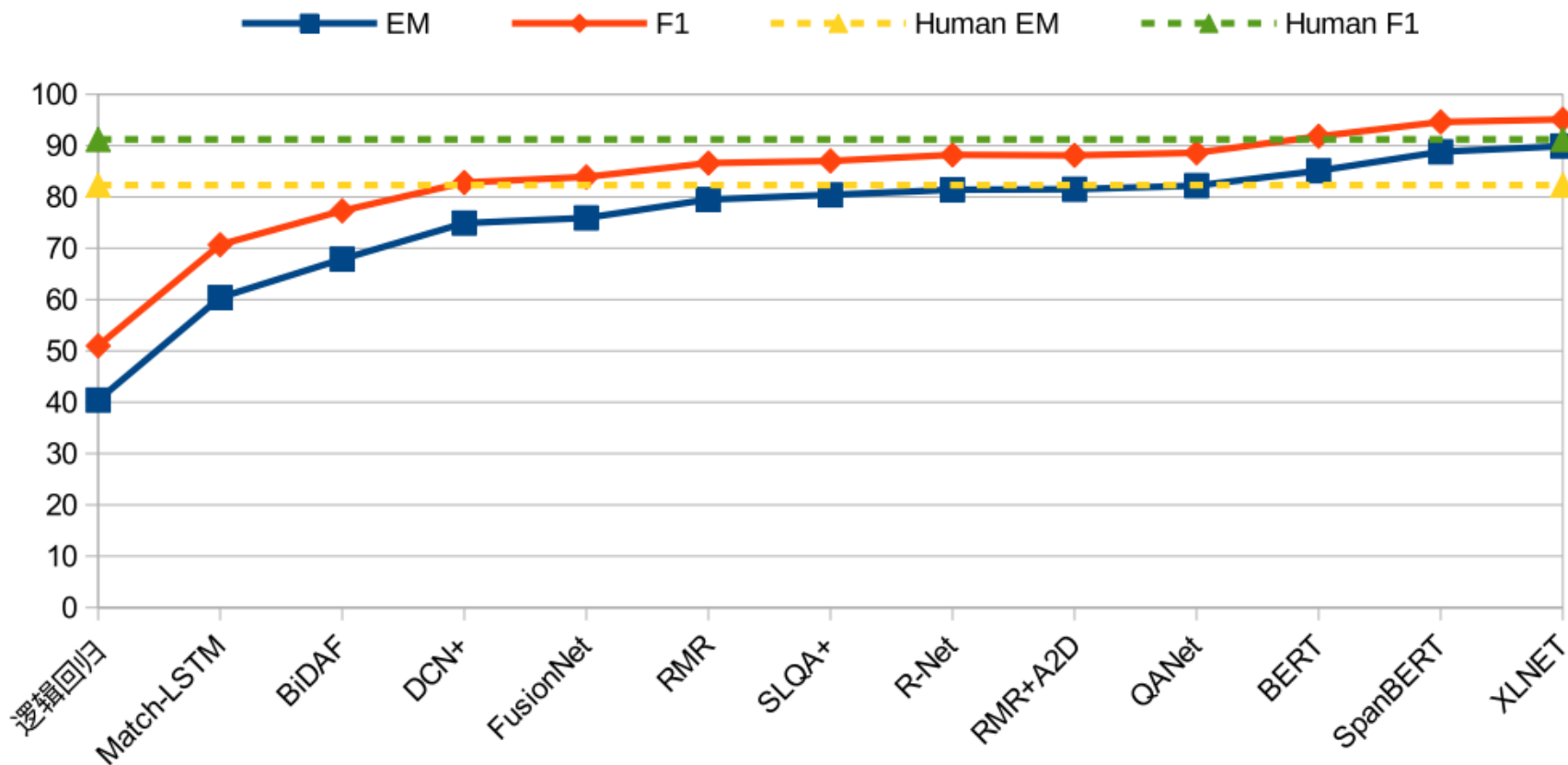
Gold answers: (1) independent (2) independent schools (3) independent schools

1) **System answers:** independent; **bag-of-words:** {independent} Precision = 1/ 1 = 1; Recall: 1 / 1 = 1; F1 = 2 / 2 = 1

2) **System answers:** independent; **bag-of-words:** {independent, schools} Precision = 1 / 1 = 1; Recall: 1 / 2 = 0.5; F1 = (2*1*0.5) / 1.5 = 0.67

机器阅读理解

- Stanford Question Answering Dataset (SQuAD)



<https://rajpurkar.github.io/SQuAD-explorer/>

机器阅读理解

- Stanford Question Answering Dataset (SQuAD)
 - leaderboard

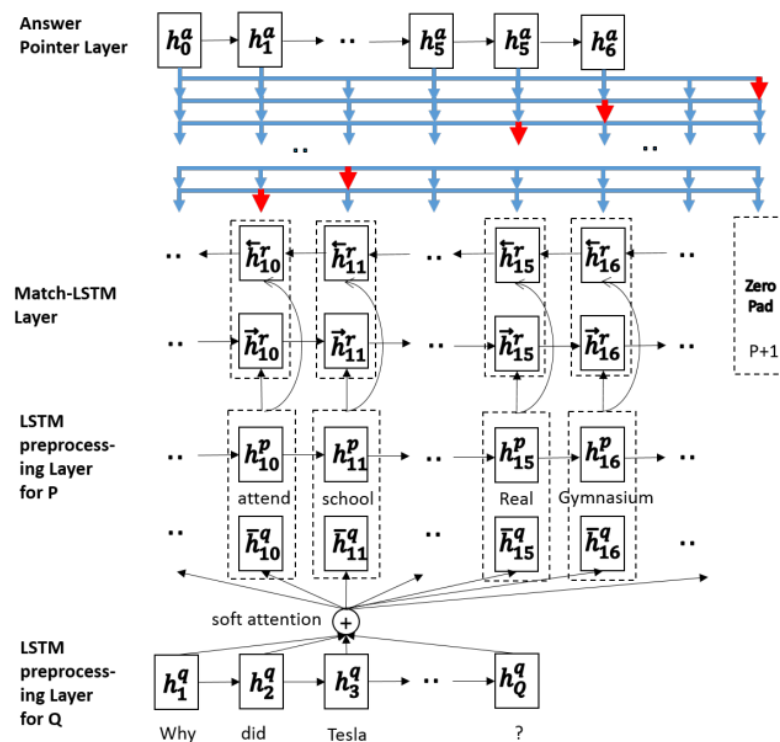
Rank	Model	EM	F1
	Human Performance Stanford University (Rajpurkar et al. '16)	82.304	91.221
1 Oct 05, 2018	BERT (ensemble) Google AI Language https://arxiv.org/abs/1810.04805	87.433	93.160
2 Oct 05, 2018	BERT (single model) Google AI Language https://arxiv.org/abs/1810.04805	85.083	91.835
2 Sep 09, 2018	nlnet (ensemble) Microsoft Research Asia	85.356	91.202
2 Sep 26, 2018	nlnet (ensemble) Microsoft Research Asia	85.954	91.677
3 Jul 11, 2018	QANet (ensemble) Google Brain & CMU	84.454	90.490
4 Jul 08, 2018	r-net (ensemble) Microsoft Research Asia	84.003	90.147
5 Mar 19, 2018	QANet (ensemble) Google Brain & CMU	83.877	89.737
5 Sep 09, 2018	nlnet (single model) Microsoft Research Asia	83.468	90.133

Rank	Model	EM	F1
	Human Performance Stanford University (Rajpurkar et al. '16)	82.304	91.221
1 Jul 24, 2021	[ANNA] (single model) LG AI Research	90.622	95.719
2 Apr 10, 2020	LUKE (single model) Studio Ousia & NAIST & RIKEN AIP https://arxiv.org/abs/2010.01057	90.202	95.379
3 May 21, 2019	XLNet (single model) Google Brain & CMU	89.898	95.080
4 Dec 11, 2019	XLNET-123++ (single model) MST/EOI http://tia.today	89.856	94.903
4 Aug 11, 2019	XLNET-123 (single model) MST/EOI	89.646	94.930
5 Jul 21, 2019	SpanBERT (single model) FAIR & UW	88.839	94.635
6 Jul 03, 2019	BERT+WWM+MT (single model) Xiao Research	88.650	94.393
7 Jul 21, 2019	Tuned BERT-1seq Large Cased (single model) FAIR & UW	87.465	93.294
8 Oct 05, 2018	BERT (ensemble) Google AI Language https://arxiv.org/abs/1810.04805	87.433	93.160
9 May 14, 2019	ATB (single model) Anonymous	86.940	92.641

<https://rajpurkar.github.io/SQuAD-explorer/>

机器阅读理解

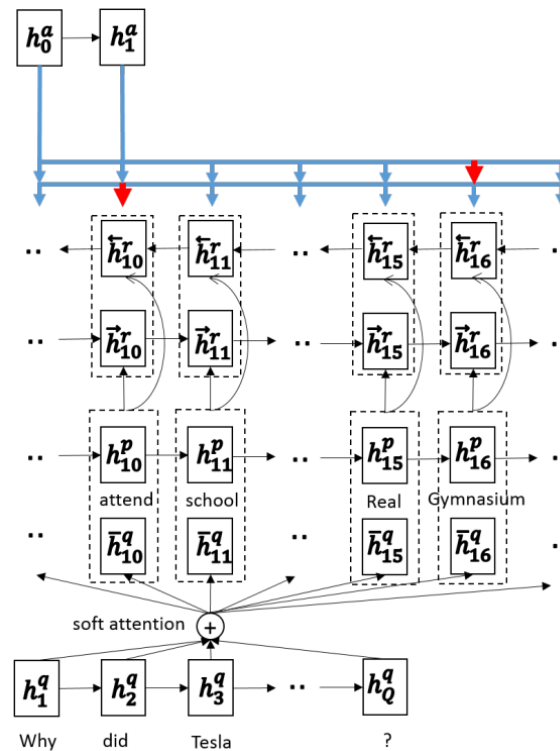
- Stanford Question Answering Dataset (SQuAD)
 - Match-LSTM
 - Sequence Model



Shuohang, WANG and Jing JIANG. Machine comprehension using match-LSTM and answer pointer. (2017). ICLR 2017

机器阅读理解

- Stanford Question Answering Dataset (SQuAD)
 - Match-LSTM
 - Boundary Model



Shuohang, WANG and Jing JIANG. Machine comprehension using match-LSTM and answer pointer. (2017). ICLR 2017

机器阅读理解

- Stanford Question Answering Dataset (SQuAD)
 - Match-LSTM
 - 实验结果

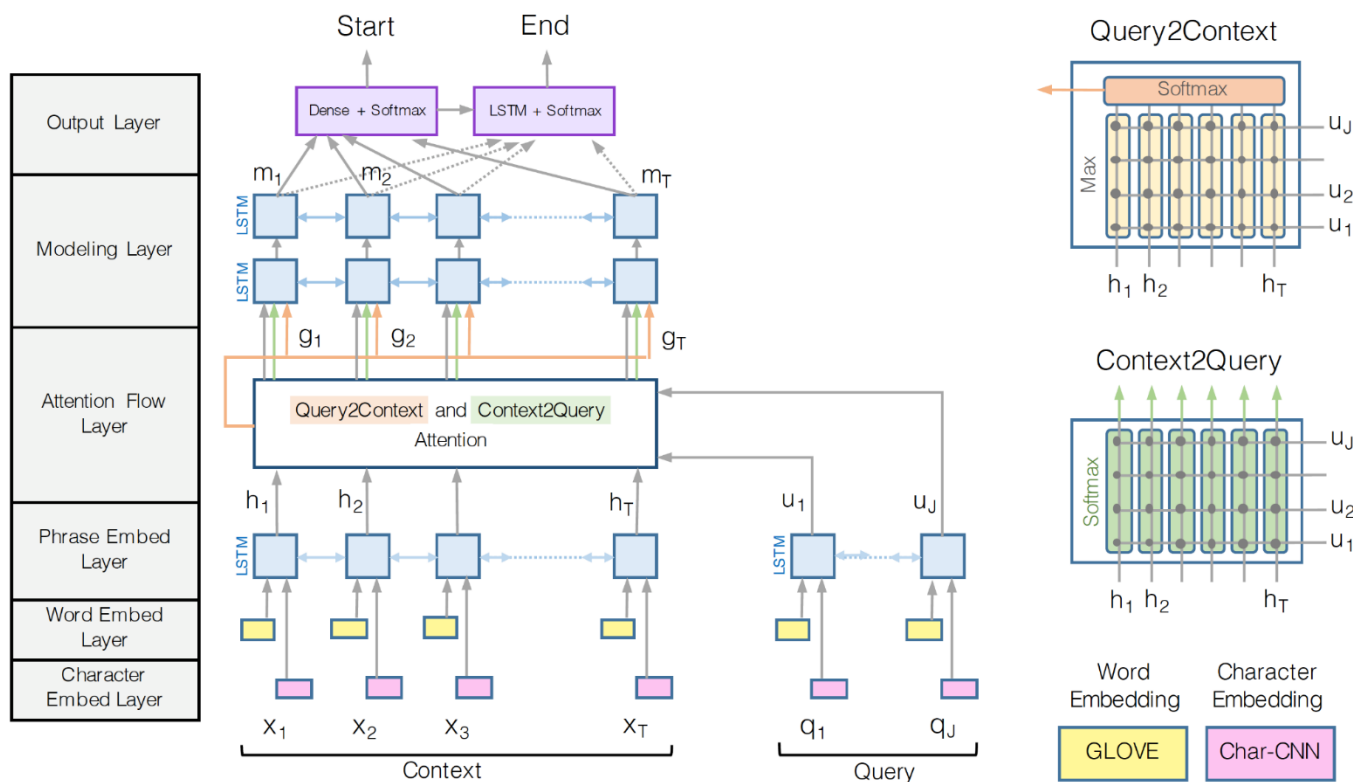
	l	$ \theta $	Exact Match		F1	
			Dev	Test	Dev	Test
Random Guess	-	0	1.1	1.3	4.1	4.3
Logistic Regression	-	-	40.0	40.4	51.0	51.0
DCR	-	-	62.5	62.5	71.2	71.0
Match-LSTM with Ans-Ptr (Sequence)	150	882K	54.4	-	68.2	-
Match-LSTM with Ans-Ptr (Boundary)	150	882K	61.1	-	71.2	-
Match-LSTM with Ans-Ptr (Boundary+Search)	150	882K	63.0	-	72.7	-
Match-LSTM with Ans-Ptr (Boundary+Search)	300	3.2M	63.1	-	72.7	-
Match-LSTM with Ans-Ptr (Boundary+Search+b)	150	1.1M	63.4	-	73.0	-
Match-LSTM with Bi-Ans-Ptr (Boundary+Search+b)	150	1.4M	64.1	64.7	73.9	73.7
Match-LSTM with Ans-Ptr (Boundary+Search+en)	150	882K	67.6	67.9	76.8	77.0

Table 2: Experiment Results. Here “Search” refers to globally searching the spans with no more than 15 tokens, “b” refers to using bi-directional pre-processing LSTM, and “en” refers to ensemble method.

Shuohang, WANG and Jing JIANG. Machine comprehension using match-LSTM and answer pointer. (2017). ICLR 2017

机器阅读理解

- Stanford Question Answering Dataset (SQuAD)
 - Bi-Directional Attention Flow (BiDAF)



Seo, Minjoon, Aniruddha Kembhavi, Ali Farhadi, and Hannaneh Hajishirzi. "Bidirectional attention flow for machine comprehension." (2017). ICLR 2017

机器阅读理解

- Stanford Question Answering Dataset (SQuAD)
 - Bi-Directional Attention Flow (BiDAF)
 - 实验结果

	Single Model		Ensemble			EM	F1
	EM	F1	EM	F1		EM	F1
Logistic Regression Baseline ^a	40.4	51.0	-	-	No char embedding	65.0	75.4
Dynamic Chunk Reader ^b	62.5	71.0	-	-	No word embedding	55.5	66.8
Fine-Grained Gating ^c	62.5	73.3	-	-	No C2Q attention	57.2	67.7
Match-LSTM ^d	64.7	73.7	67.9	77.0	No Q2C attention	63.6	73.7
Multi-Perspective Matching ^e	65.5	75.1	68.2	77.2	Dynamic attention	63.5	73.6
Dynamic Coattention Networks ^f	66.2	75.9	71.6	80.4	BiDAF (single)	67.7	77.3
R-Net ^g	68.4	77.5	72.1	79.7	BiDAF (ensemble)	72.6	80.7
BiDAF (Ours)	68.0	77.3	73.3	81.1			

(a) Results on the SQuAD test set

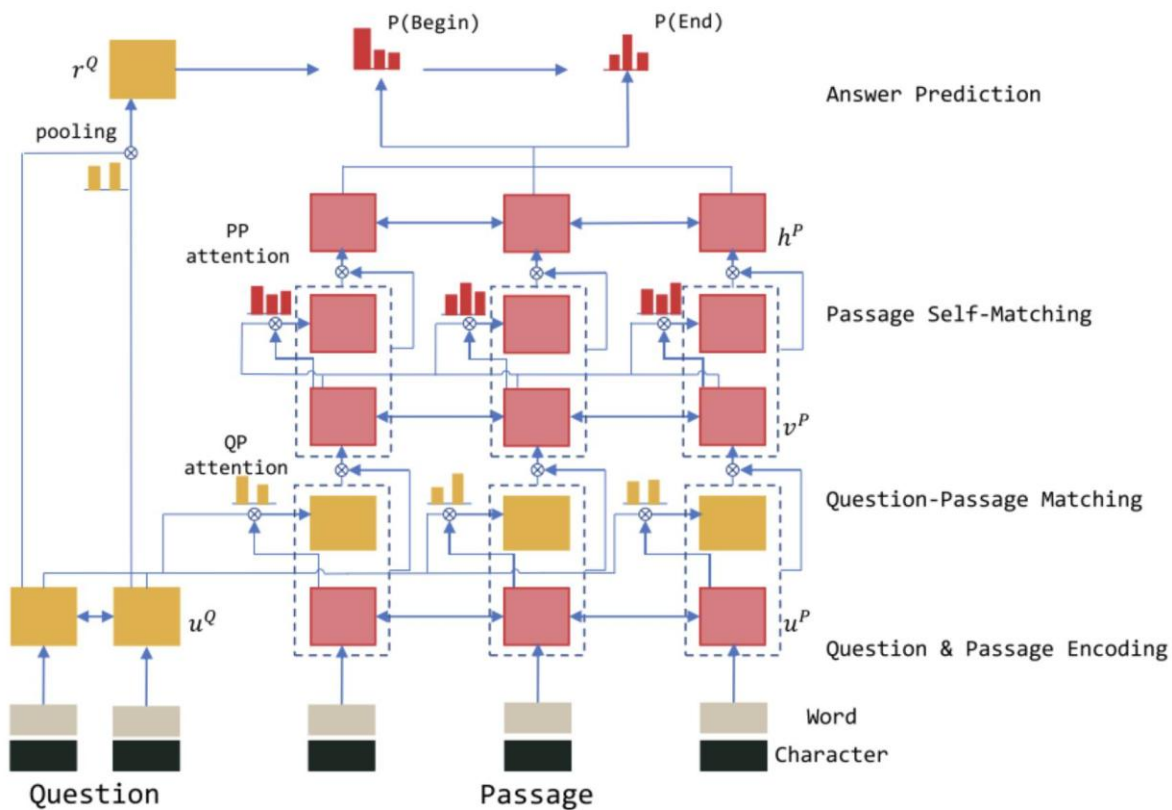
(b) Ablations on the SQuAD dev set

Table 1: (1a) The performance of our model BiDAF and competing approaches by Rajpurkar et al. (2016)^a, Yu et al. (2016)^b, Yang et al. (2016)^c, Wang & Jiang (2016)^d, IBM Watson^e (unpublished), Xiong et al. (2016b)^f, and Microsoft Research Asia^g (unpublished) on the SQuAD test set. A concurrent work by Lee et al. (2016) does not report the test scores. All results shown here reflect the SQuAD leaderboard (stanford-qa.com) as of 6 Dec 2016, 12pm PST. (1b) The performance of our model and its ablations on the SQuAD dev set. Ablation results are presented only for single runs.

Seo, Minjoon, Aniruddha Kembhavi, Ali Farhadi, and Hannaneh Hajishirzi. "Bidirectional attention flow for machine comprehension." (2017). ICLR 2017

机器阅读理解

- Stanford Question Answering Dataset (SQuAD)
 - R-Net



Wang, Wenhui, Nan Yang, Furu Wei, Baobao Chang, and Ming Zhou. "Gated self-matching networks for reading comprehension and question answering." In Proceedings of ACL, pp. 189-198. 2017.

机器阅读理解

- Stanford Question Answering Dataset (SQuAD)

- R-Net

- 实验结果

	Dev Set	Test Set
<i>Single model</i>	EM / F1	EM / F1
LR Baseline (Rajpurkar et al., 2016)	40.0 / 51.0	40.4 / 51.0
Dynamic Chunk Reader (Yu et al., 2016)	62.5 / 71.2	62.5 / 71.0
Match-LSTM with Ans-Ptr (Wang and Jiang, 2016b)	64.1 / 73.9	64.7 / 73.7
Dynamic Coattention Networks (Xiong et al., 2016)	65.4 / 75.6	66.2 / 75.9
RaSoR (Lee et al., 2016)	66.4 / 74.9	- / -
BiDAF (Seo et al., 2016)	68.0 / 77.3	68.0 / 77.3
jNet (Zhang et al., 2017)	- / -	68.7 / 77.4
Multi-Perspective Matching (Wang et al., 2016)	- / -	68.9 / 77.8
FastQA (Weissenborn et al., 2017)	- / -	68.4 / 77.1
FastQAExt (Weissenborn et al., 2017)	- / -	70.8 / 78.9
R-NET	71.1 / 79.5	71.3 / 79.7
<i>Ensemble model</i>		
Fine-Grained Gating (Yang et al., 2016)	62.4 / 73.4	62.5 / 73.3
Match-LSTM with Ans-Ptr (Wang and Jiang, 2016b)	67.6 / 76.8	67.9 / 77.0
RaSoR (Lee et al., 2016)	68.2 / 76.7	- / -
Dynamic Coattention Networks (Xiong et al., 2016)	70.3 / 79.4	71.6 / 80.4
BiDAF (Seo et al., 2016)	73.3 / 81.1	73.3 / 81.1
Multi-Perspective Matching (Wang et al., 2016)	- / -	73.8 / 81.3
R-NET	75.6 / 82.8	75.9 / 82.9
Human Performance (Rajpurkar et al., 2016)	80.3 / 90.5	77.0 / 86.8

Table 2: The performance of our gated self-matching networks (R-NET) and competing approaches⁴.

Single Model	EM / F1
Gated Self-Matching (GRU)	71.1 / 79.5
-Character embedding	69.6 / 78.6
-Gating	67.9 / 77.1
-Self-Matching	67.6 / 76.7
-Gating, -Self-Matching	65.4 / 74.7

Table 3: Ablation tests of single model on the SQuAD dev set. All the components significantly (t-test, $p < 0.05$) improve the model.

Single Model	EM / F1
Base model (GRU)	64.5 / 74.1
+Gating	66.2 / 75.8
Base model (LSTM)	64.2 / 73.9
+Gating	66.0 / 75.6

Table 4: Effectiveness of gated attention-based recurrent networks for both GRU and LSTM.

Wang, Wenhui, Nan Yang, Furu Wei, Baobao Chang, and Ming Zhou. "Gated self-matching networks for reading comprehension and question answering." In Proceedings of ACL, pp. 189-198. 2017.

机器阅读理解

- Stanford Question Answering Dataset V2.0 (SQuAD 2.0, Rajpurkar et al., 2018)
 - SQuAD 1.0缺点
 - 所有问题都能从文章中找到答案
 - SQuAD 2.0
 - 训练集中1/3问题没有答案，而验证或者测试集中1/2问题没有答案
 - 对于没有答案的问题，只有回答出no answer是EM和F1 score才为1，其他情况EM和F1 score均为0

机器阅读理解

- Stanford Question Answering Dataset V2.0 (SQuAD 2.0)

Genghis Khan united the Mongol and Turkic tribes of the steppes and became Great Khan in 1206. He and his successors expanded the Mongol empire across Asia. Under the reign of Genghis' third son, Ögedei Khan, the Mongols destroyed the weakened Jin dynasty in 1234, conquering most of northern China. Ögedei offered his nephew Kublai a position in Xingzhou, Hebei. Kublai was unable to read Chinese but had several Han Chinese teachers attached to him since his early years by his mother Sorghaghtani. He sought the counsel of Chinese Buddhist and Confucian advisers. Möngke Khan succeeded Ögedei's son, Güyük, as Great Khan in 1251. He

When did Genghis Khan kill Great Khan?

Gold Answers: <No Answer>

Prediction: 1234 [from Microsoft nlnet]

机器阅读理解

- Stanford Question Answering Dataset V2.0 (SQuAD 2.0)
 - leaderboard

Rank	Model	EM	F1
	Human Performance Stanford University (Rajpurkar & Jia et al. '18)	86.831	89.452
1 Jun 04, 2021	IE-Net (ensemble) RICOH_SRCB_DML	90.939	93.214
2 Feb 21, 2021	FPNet (ensemble) Ant Service Intelligence Team	90.871	93.183
3 May 16, 2021	IE-NetV2 (ensemble) RICOH_SRCB_DML	90.860	93.100
4 Apr 06, 2020	SA-Net on Albert (ensemble) QIANXIN	90.724	93.011
5 May 05, 2020	SA-Net-V2 (ensemble) QIANXIN	90.679	92.948
5 Apr 05, 2020	Retro-Reader (ensemble) Shanghai Jiao Tong University http://arxiv.org/abs/2001.09694	90.578	92.978
5 Feb 05, 2021	FPNet (ensemble) YuYang	90.600	92.899
6 Apr 18, 2021	TransNets + SFVerifier + SFEnsembler (ensemble) Senseforth AI Research https://www.senseforth.ai/	90.487	92.894
6 Dec 01, 2020	EntitySpanFocusV2 (ensemble) RICOH_SRCB_DML	90.521	92.824
6 Jul 31, 2020	ATRLP+PV (ensemble) Hithink RoyalFlush	90.442	92.877
7 Mar 12, 2020	ALBERT + DAAF + Verifier (ensemble) PINGAN Omni-Sinitic	90.386	92.777

82 Jul 13, 2018	VS^3-NET (single model) Kangwon National University in South Korea	67.897	70.884
83 Jan 01, 2019	EBB-Net (single model) Enliple AI	66.610	70.303
84 Jun 25, 2018	KakaoNet2 (single model) Kakao NLP Team	65.719	69.381
85 Sep 13, 2018	BiDAF++ (single model) UW and FAIR	65.651	68.866
85 Jul 11, 2018	abcNet (single model) Fudan University & Liulishuo AI Lab	65.256	69.206
86 Jun 27, 2018	BSAE AddText (single model) reciTAL.ai	63.338	67.422
87 Aug 14, 2018	eeAttNet (single model) BBD NLP Team https://www.bbdservice.com	63.327	66.633
87 May 30, 2018	BiDAF + Self Attention + ELMo (single model) Allen Institute for Artificial Intelligence [modified by Stanford]	63.372	66.251
88 May 30, 2018	BiDAF + Self Attention (single model) Allen Institute for Artificial Intelligence [modified by Stanford]	59.332	62.305
89 May 30, 2018	BiDAF-No-Answer (single model) University of Washington [modified by Stanford]	59.174	62.093
89 Nov 27, 2018	Tree-LSTM + BiDAF + ELMo (single model) Carnegie Mellon University	57.707	62.341

机器阅读理解

- Open-Domain Question Answering (Chen, et al. 2017)

Q: How many of Warsaw's inhabitants spoke Polish in 1933?



循环神经网络的应用

- 引言
- 循环神经网络
- 参数学习
- 长程依赖问题
- 基于门控的循环神经网络
- 深度循环神经网络
- 递归神经网络
- 注意力机制
- 循环神经网络以及注意力机制的应用
 - 序列标注任务
 - 机器阅读理解
 - 自然语言推断、文本生成、看图说话等

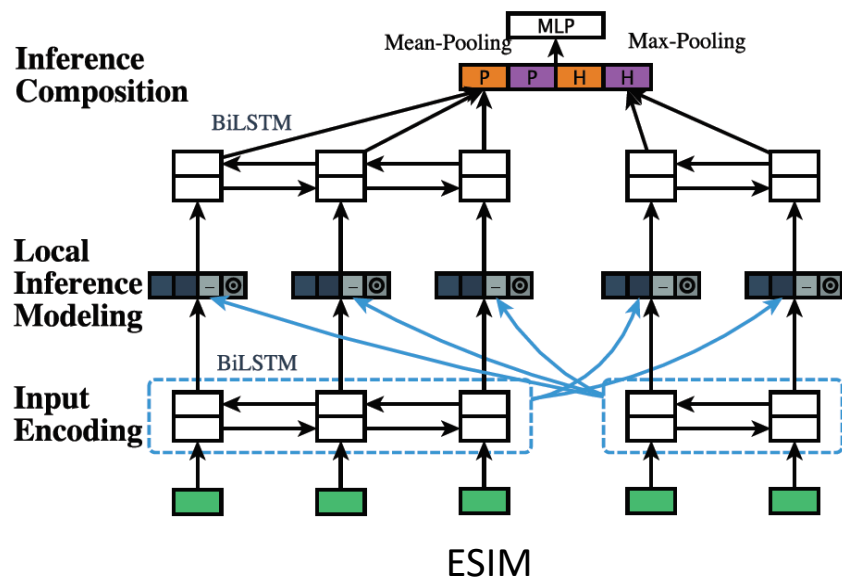
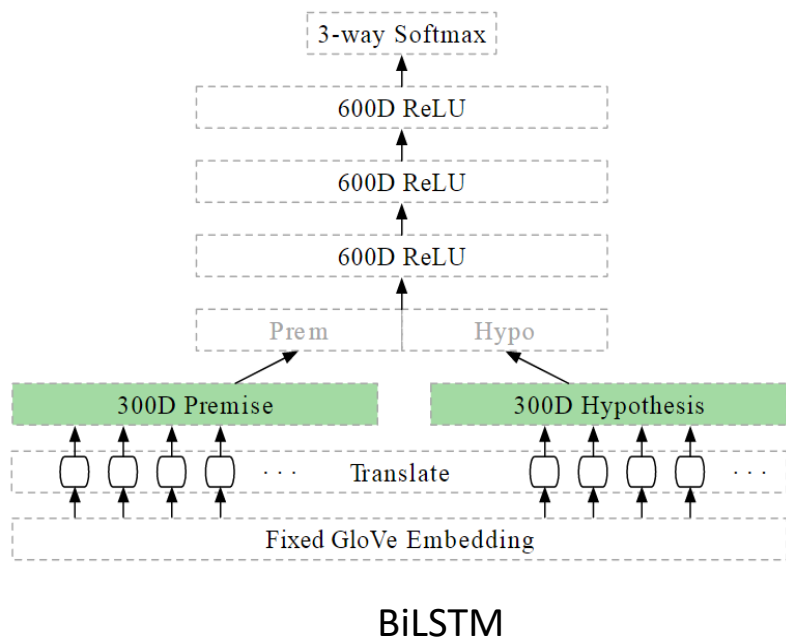
自然语言推断

- 自然语言推断（Natural Language Inference）
 - 输入为两个句子
 - 前提（Premise）
 - 假设（Hypothesis）
 - 输出为两个句子之间的关系
 - 蕴含（entailment）
 - 对立（contradiction）
 - 中立（neutral）

Text	Judgments	Hypothesis
A man inspects the uniform of a figure in some East Asian country.	contradiction C C C C C	The man is sleeping
An older and younger man smiling.	neutral N N E N N	Two men are smiling and laughing at the cats playing on the floor.
A black race car starts up in front of a crowd of people.	contradiction C C C C C	A man is driving down a lonely road.
A soccer game with multiple males playing.	entailment E E E E E	Some men are playing a sport.
A smiling costumed woman is holding an umbrella.	neutral N N E C N	A happy woman in a fairy costume holds an umbrella.

自然语言推断

- 典型模型



代码注释或代码生成

- Python代码注释生成示例

```
def column_summation(y_out, x_lengths, mode="mean"):
    '''Get a max or mean vector from each batch datapoint in `y_out`.
    More precisely, iterate over batch row indices, get the max or mean vector of all the vectors by
    the position indicated by the corresponding value in `x_lengths` at the row index.
    Args:
        y_out (torch.FloatTensor, torch.cuda.FloatTensor)
            shape: (batch, sequence, feature)
        x_lengths (torch.LongTensor, torch.cuda.LongTensor)
            shape: (batch,)
        mode: "mean" for mean vector; "max" for max vector
    Returns:
        y_out (torch.FloatTensor, torch.cuda.FloatTensor)
            shape: (batch, feature)
        ...
    x_lengths = x_lengths.long().detach().cpu().numpy() - 1
    out = []
    for batch_index, column_index in enumerate(x_lengths):
        if mode == "mean":
            avg = torch.mean(y_out[batch_index, :column_index],0)
            out.append(avg)
        elif mode == "max":
            maximum,maximum_index = torch.max(y_out[batch_index, :column_index],0)
            out.append(maximum)
        else:
            out.append(y_out[batch_index, column_index])
    return torch.stack(out)
```

作词机

- RNN在“学习”过汪峰全部作品后自动生成的歌词
 - <https://github.com/phunterlau/wangfeng-rnn>

我在这里中的夜里
就像一场是一种生命的意叶
就像我的生活变得在我一样
可我们这是一个知道
我只是一天你会怎吗
可我们这是我们是不要为你
我们想这有一种生活的时候

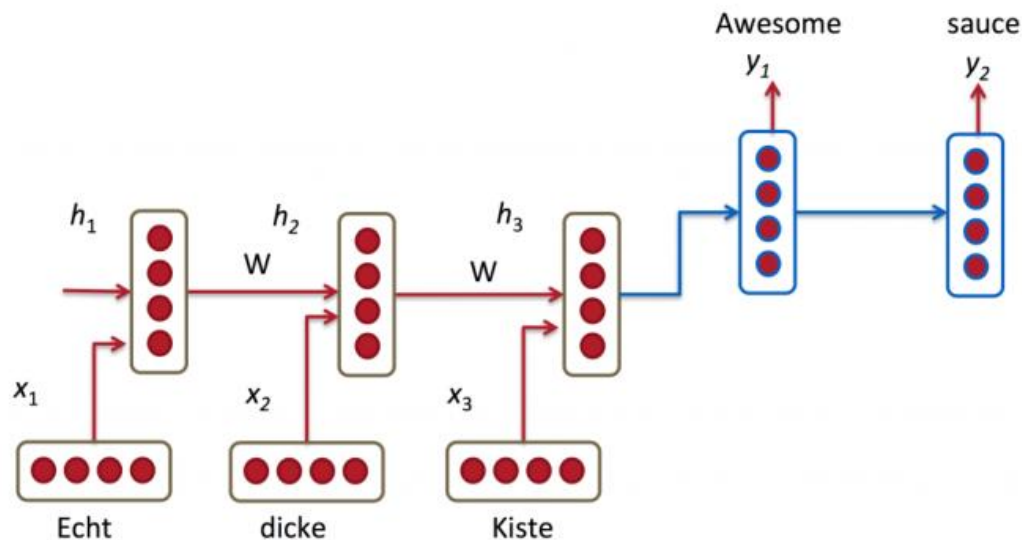
作诗

- RNN在“学习”过所有的唐诗宋词

白鹭窥鱼立, Egrets stood, peeping fishes. 青山照水开. Water was still, reflecting mountains. 夜来风不动, The wind went down by nightfall, 明月见楼台. as the moon came up by the tower.	满怀风月一枝春, Budding branches are full of romance. 未见梅花亦可人. Plum blossoms are invisible but adorable. 不为东风无此客, With the east wind comes Spring. 世间何处是前身. Where on earth do I come from?
--	--

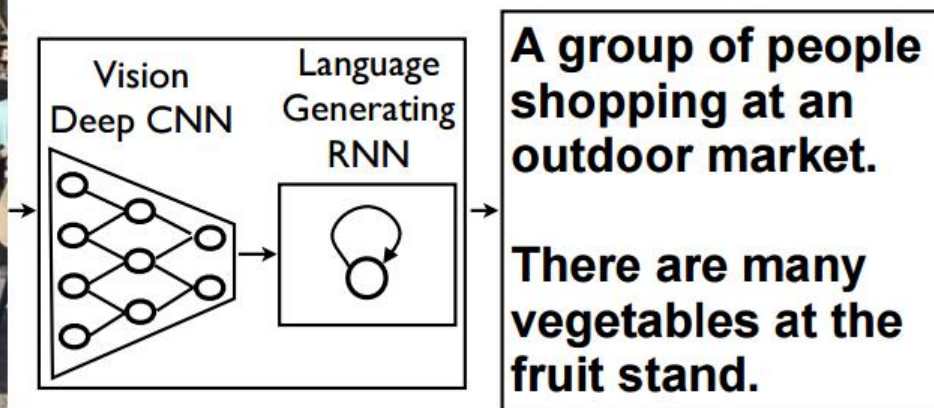
基于序列到序列的机器翻译

- 序列到序列模型 (Sequence-to-Sequence)
 - 一个RNN用来编码
 - 另一个RNN用来解码



看图说话

- Image Captioning



看图说话

- Image Captioning

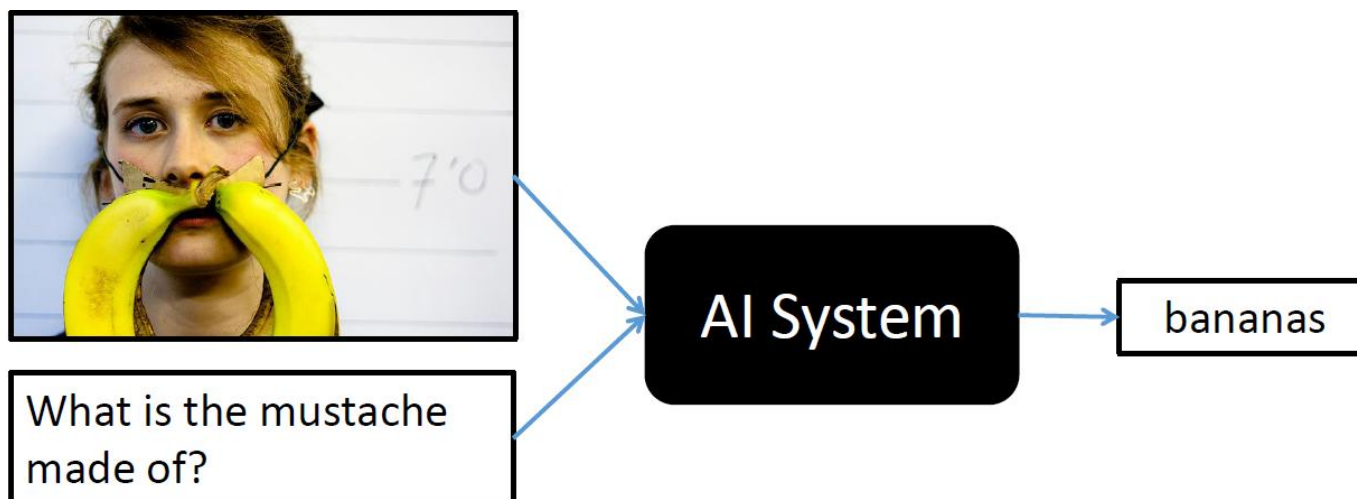


Figure 5. A selection of evaluation results, grouped by human rating.

Visual Question Answering (VQA)

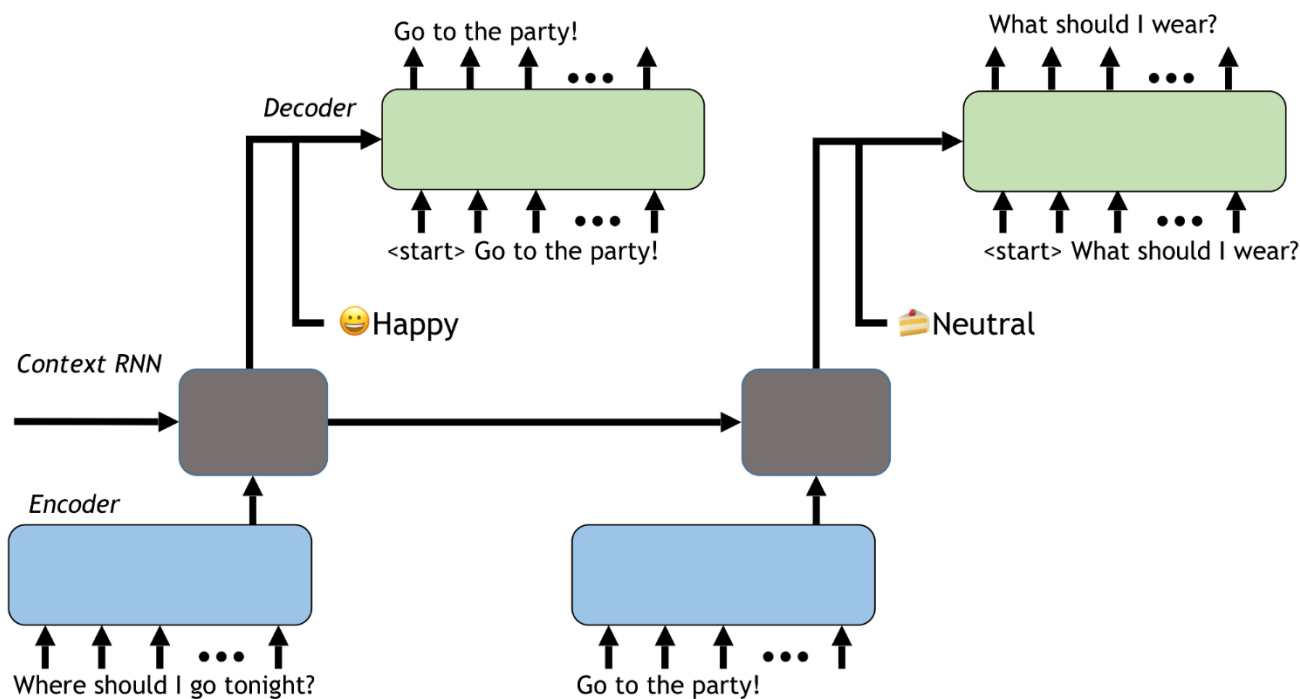
- 基于视觉的问答

- Given an image and a natural language question about the image, the task is to provide an accurate natural language answer



对话系统

- Dialogue System



<https://github.com/lukalabs/cakechat>

课程作业

- 基于循环神经网络的命名实体识别
 - 问题描述
 - 利用循环神经网络，识别出CoNLL-2003数据集的实体以及类别
 - 数据集
 - CONLL 2003
 - 链接: <https://www.clips.uantwerpen.be/conll2003/ner/>
 - 要求
 - Tensorflow或者Pytorch实现
 - 评价指标
 - Precision、Recall、F1
 - 参考资料
 - 《神经网络与深度学习》 第6、11章
 - <https://arxiv.org/pdf/1603.01354.pdf>
 - <https://arxiv.org/pdf/1603.01360.pdf>



欢迎提问！